

RMW desert

5.0.0

Generated by Doxygen 1.15.0

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Namespace Documentation	9
5.1 CStringHelper Namespace Reference	9
5.1.1 Detailed Description	10
5.1.2 Function Documentation	10
5.1.2.1 assign_string()	10
5.1.2.2 assign_u16string()	10
5.1.2.3 assign_vector_string()	11
5.1.2.4 assign_vector_string_to_sequence()	11
5.1.2.5 assign_vector_u16string()	11
5.1.2.6 assign_vector_u16string_to_sequence()	12
5.1.2.7 convert_sequence_to_std_vector_string()	12
5.1.2.8 convert_sequence_to_std_vector_u16string()	13
5.1.2.9 convert_to_std_string()	13
5.1.2.10 convert_to_std_u16string()	13
5.1.2.11 convert_to_std_vector_string()	14
5.1.2.12 convert_to_std_vector_u16string()	14
5.2 Discovery Namespace Reference	14
5.2.1 Detailed Description	15
5.2.2 Function Documentation	16
5.2.2.1 demangle_publisher_from_topic()	16
5.2.2.2 demangle_service_from_topic()	16
5.2.2.3 demangle_service_reply_from_topic()	16
5.2.2.4 demangle_service_request_from_topic()	17
5.2.2.5 demangle_subscriber_from_topic()	17
5.2.2.6 demangle_topic()	18
5.2.2.7 discovery_thread()	18
5.2.2.8 discovery_thread_start()	18
5.2.2.9 discovery_thread_stop()	19
5.2.2.10 identity_demangle()	19
5.2.2.11 resolve_prefix()	19
5.2.2.12 send_discovery_beacon()	20
5.2.2.13 send_discovery_request()	20

5.3 MessageSerialization Namespace Reference	21
5.3.1 Detailed Description	21
5.3.2 Function Documentation	21
5.3.2.1 deserialize()	21
5.3.2.2 deserialize_field() [1/2]	22
5.3.2.3 deserialize_field() [2/2]	22
5.3.2.4 serialize()	23
5.3.2.5 serialize_field() [1/2]	23
5.3.2.6 serialize_field() [2/2]	24
6 Class Documentation	25
6.1 security::AeadParams Class Reference	25
6.1.1 Constructor & Destructor Documentation	25
6.1.1.1 AeadParams()	25
6.1.2 Member Function Documentation	26
6.1.2.1 get_alg()	26
6.1.2.2 get_key_size()	26
6.1.2.3 get_nonce_size()	26
6.1.2.4 get_tag_size()	26
6.2 CircularQueue< T, MaxLen, Container > Class Template Reference	27
6.3 DesertClient Class Reference	28
6.3.1 Constructor & Destructor Documentation	28
6.3.1.1 DesertClient()	28
6.3.2 Member Function Documentation	28
6.3.2.1 get_gid()	28
6.3.2.2 get_request_type_name()	29
6.3.2.3 get_response_type_name()	29
6.3.2.4 get_service_name()	29
6.3.2.5 has_data()	29
6.3.2.6 read_response()	30
6.3.2.7 send_request()	30
6.4 DesertGuardCondition Class Reference	30
6.4.1 Member Function Documentation	31
6.4.1.1 get_has_triggered()	31
6.4.1.2 has_triggered()	31
6.4.1.3 trigger()	31
6.5 DesertNode Class Reference	32
6.5.1 Constructor & Destructor Documentation	32
6.5.1.1 DesertNode()	32
6.5.2 Member Function Documentation	33
6.5.2.1 add_client()	33
6.5.2.2 add_publisher()	33

6.5.2.3 add_service()	33
6.5.2.4 add_subscriber()	34
6.5.2.5 get_gid()	34
6.5.2.6 remove_client()	34
6.5.2.7 remove_publisher()	35
6.5.2.8 remove_service()	35
6.5.2.9 remove_subscriber()	35
6.6 DesertPublisher Class Reference	36
6.6.1 Constructor & Destructor Documentation	36
6.6.1.1 DesertPublisher()	36
6.6.2 Member Function Documentation	36
6.6.2.1 get_gid()	36
6.6.2.2 get_topic_name()	37
6.6.2.3 get_type_name()	37
6.6.2.4 push()	37
6.7 DesertService Class Reference	38
6.7.1 Constructor & Destructor Documentation	38
6.7.1.1 DesertService()	38
6.7.2 Member Function Documentation	38
6.7.2.1 get_gid()	38
6.7.2.2 get_request_type_name()	39
6.7.2.3 get_response_type_name()	39
6.7.2.4 get_service_name()	39
6.7.2.5 has_data()	39
6.7.2.6 read_request()	40
6.7.2.7 send_response()	40
6.8 DesertSubscriber Class Reference	40
6.8.1 Constructor & Destructor Documentation	41
6.8.1.1 DesertSubscriber()	41
6.8.2 Member Function Documentation	41
6.8.2.1 get_gid()	41
6.8.2.2 get_topic_name()	41
6.8.2.3 get_type_name()	42
6.8.2.4 has_data()	42
6.8.2.5 read_data()	42
6.9 DesertWaitset Class Reference	43
6.10 cbor::TxStream::ERROR Struct Reference	43
6.11 GenericCSequence< T > Struct Template Reference	43
6.12 cbor::RxStream::ITEM Struct Reference	43
6.13 rmw_context_impl_s Struct Reference	44
6.14 cbor::RxStream Class Reference	44
6.14.1 Constructor & Destructor Documentation	45

6.14.1.1 RxStream()	45
6.14.2 Member Function Documentation	46
6.14.2.1 clear_buffer()	46
6.14.2.2 data_available()	46
6.14.2.3 deserialize_sequence()	46
6.14.2.4 get_identifier()	46
6.14.2.5 get_name()	47
6.14.2.6 get_type()	47
6.14.2.7 interpret_packets()	47
6.14.2.8 operator>>() [1/16]	47
6.14.2.9 operator>>() [2/16]	48
6.14.2.10 operator>>() [3/16]	48
6.14.2.11 operator>>() [4/16]	48
6.14.2.12 operator>>() [5/16]	48
6.14.2.13 operator>>() [6/16]	49
6.14.2.14 operator>>() [7/16]	49
6.14.2.15 operator>>() [8/16]	49
6.14.2.16 operator>>() [9/16]	49
6.14.2.17 operator>>() [10/16]	50
6.14.2.18 operator>>() [11/16]	50
6.14.2.19 operator>>() [12/16]	50
6.14.2.20 operator>>() [13/16]	50
6.14.2.21 operator>>() [14/16]	51
6.14.2.22 operator>>() [15/16]	51
6.14.2.23 operator>>() [16/16]	51
6.14.2.24 push_packet()	51
6.15 security::SecurityLayer Class Reference	52
6.15.1 Constructor & Destructor Documentation	52
6.15.1.1 SecurityLayer() [1/2]	52
6.15.1.2 SecurityLayer() [2/2]	52
6.15.2 Member Function Documentation	53
6.15.2.1 unwrap()	53
6.15.2.2 wrap()	54
6.16 TcpDaemon Class Reference	54
6.16.1 Member Function Documentation	55
6.16.1.1 enqueue_packet()	55
6.16.1.2 init()	55
6.16.1.3 read_packet()	55
6.17 TopicsConfig Class Reference	56
6.17.1 Member Function Documentation	56
6.17.1.1 get_identifier_topic()	56
6.17.1.2 get_topic_identifier()	56

6.17.1.3 load_configuration()	56
6.18 cbor::TxStream Class Reference	57
6.18.1 Constructor & Destructor Documentation	58
6.18.1.1 TxStream()	58
6.18.2 Member Function Documentation	58
6.18.2.1 end_transmission()	58
6.18.2.2 operator<<() [1/16]	58
6.18.2.3 operator<<() [2/16]	59
6.18.2.4 operator<<() [3/16]	59
6.18.2.5 operator<<() [4/16]	59
6.18.2.6 operator<<() [5/16]	59
6.18.2.7 operator<<() [6/16]	60
6.18.2.8 operator<<() [7/16]	60
6.18.2.9 operator<<() [8/16]	60
6.18.2.10 operator<<() [9/16]	60
6.18.2.11 operator<<() [10/16]	61
6.18.2.12 operator<<() [11/16]	61
6.18.2.13 operator<<() [12/16]	61
6.18.2.14 operator<<() [13/16]	61
6.18.2.15 operator<<() [14/16]	62
6.18.2.16 operator<<() [15/16]	62
6.18.2.17 operator<<() [16/16]	62
6.18.2.18 serialize_sequence()	62
6.18.2.19 start_transmission() [1/2]	63
6.18.2.20 start_transmission() [2/2]	63
6.19 cbor::TxStream::WRITER Struct Reference	63
7 File Documentation	65
7.1 src/desert_classes/CBorStream.h File Reference	65
7.1.1 Detailed Description	66
7.2 CBorStream.h	67
7.3 src/desert_classes/CStringHelper.h File Reference	69
7.3.1 Detailed Description	71
7.4 CStringHelper.h	71
7.5 src/desert_classes/demangle.h File Reference	72
7.5.1 Detailed Description	73
7.6 demangle.h	74
7.7 src/desert_classes/DesertClient.h File Reference	74
7.7.1 Detailed Description	75
7.8 DesertClient.h	76
7.9 src/desert_classes/DesertGuardCondition.h File Reference	77
7.9.1 Detailed Description	77

7.10	DesertGuardCondition.h	77
7.11	src/desert_classes/DesertNode.h File Reference	78
7.11.1	Detailed Description	78
7.12	DesertNode.h	79
7.13	src/desert_classes/DesertPublisher.h File Reference	80
7.13.1	Detailed Description	81
7.14	DesertPublisher.h	81
7.15	src/desert_classes/DesertService.h File Reference	82
7.15.1	Detailed Description	83
7.16	DesertService.h	83
7.17	src/desert_classes/DesertSubscriber.h File Reference	84
7.17.1	Detailed Description	85
7.18	DesertSubscriber.h	86
7.19	src/desert_classes/DesertWaitSet.h File Reference	87
7.19.1	Detailed Description	87
7.20	DesertWaitSet.h	87
7.21	src/desert_classes/Discovery.h File Reference	88
7.21.1	Detailed Description	89
7.22	Discovery.h	89
7.23	src/desert_classes/macros.h File Reference	90
7.23.1	Detailed Description	90
7.23.2	Macro Definition Documentation	91
7.23.2.1	SPECIALIZE_GENERIC_C_SEQUENCE	91
7.24	macros.h	91
7.25	src/desert_classes/MessageSerialization.h File Reference	92
7.25.1	Detailed Description	93
7.26	MessageSerialization.h	94
7.27	src/desert_classes/rmw_context_impl_s.h File Reference	98
7.27.1	Detailed Description	99
7.28	rmw_context_impl_s.h	99
7.29	src/desert_classes/security/AeadParams.h File Reference	100
7.29.1	Detailed Description	101
7.30	AeadParams.h	101
7.31	src/desert_classes/security/sec_utils.h File Reference	102
7.31.1	Detailed Description	102
7.31.2	Macro Definition Documentation	103
7.31.2.1	MIN	103
7.31.3	Function Documentation	103
7.31.3.1	decode_hex()	103
7.32	sec_utils.h	103
7.33	src/desert_classes/security/SecurityLayer.h File Reference	104
7.33.1	Detailed Description	105

7.34 SecurityLayer.h	105
7.35 src/desert_classes/TcpDaemon.h File Reference	106
7.35.1 Detailed Description	107
7.36 TcpDaemon.h	108
7.37 src/desert_classes/TopicsConfig.h File Reference	108
7.37.1 Detailed Description	109
7.38 TopicsConfig.h	110
Index	111

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

CStringHelper	
Namespace containing C sequence handling functions	9
Discovery	
Namespace containing discovery functions	14
MessageSerialization	
Namespace containing serialization functions	21

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

security::AeadParams	25
DesertClient	28
DesertGuardCondition	30
DesertNode	32
DesertPublisher	36
DesertService	38
DesertSubscriber	40
DesertWaitset	43
cbor::TxStream::ERROR	43
GenericCSequence< T >	43
cbor::RxStream::ITEM	43
std::queue	
CircularQueue< T, MaxLen, Container >	27
std::queue< std::vector< std::pair< void *, int > >, std::deque< std::vector< std::pair< void *, int > > > >	
CircularQueue< std::vector< std::pair< void *, int > >, MAX_BUFFER_CAPACITY >	27
rmw_context_impl_s	44
cbor::RxStream	44
security::SecurityLayer	52
TcpDaemon	54
TopicsConfig	56
cbor::TxStream	57
cbor::TxStream::WRITER	63

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

security::AeadParams	25
CircularQueue< T, MaxLen, Container >	27
DesertClient	28
DesertGuardCondition	30
DesertNode	32
DesertPublisher	36
DesertService	38
DesertSubscriber	40
DesertWaitset	43
cbor::TxStream::ERROR	43
GenericCSequence< T >	43
cbor::RxStream::ITEM	43
rmw_context_impl_s	44
cbor::RxStream	44
security::SecurityLayer	52
TcpDaemon	54
TopicsConfig	56
cbor::TxStream	57
cbor::TxStream::WRITER	63

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

src/desert_classes/ CBorStream.h	
Classes used to convert data types into a CBOR encoded stream	65
src/desert_classes/ CStringHelper.h	
Namespace containing C sequence handling functions	69
src/desert_classes/ demangle.h	
Functions used to demangle topic names during discovery operations	72
src/desert_classes/ DesertClient.h	
Implementation of the Client structure for DESERT	74
src/desert_classes/ DesertGuardCondition.h	
Implementation of the GuardCondition structure for DESERT	77
src/desert_classes/ DesertNode.h	
Implementation of the Node structure for DESERT	78
src/desert_classes/ DesertPublisher.h	
Implementation of the Publisher structure for DESERT	80
src/desert_classes/ DesertService.h	
Implementation of the Service structure for DESERT	82
src/desert_classes/ DesertSubscriber.h	
Implementation of the Subscriber structure for DESERT	84
src/desert_classes/ DesertWaitSet.h	
Implementation of the WaitSet structure for DESERT	87
src/desert_classes/ Discovery.h	
Namespace used to provide discovery functionalities	88
src/desert_classes/ macros.h	
Header containing C sequence macros	90
src/desert_classes/ MessageSerialization.h	
Namespace containing serialization functions	92
src/desert_classes/ rmw_context_impl_s.h	
Implementation for the context variable	98
src/desert_classes/ TcpDaemon.h	
Class used to send and receive data from the DESERT socket	106
src/desert_classes/ TopicsConfig.h	
Class used to store configurations	108
src/desert_classes/security/ AeadParams.h	
Class containing the parameters of the selected cryptography	100
src/desert_classes/security/ sec_utils.h	
Utilities for the security layer	102
src/desert_classes/security/ SecurityLayer.h	
Class used to handle the encryption of the stream	104

Chapter 5

Namespace Documentation

5.1 CStringHelper Namespace Reference

Namespace containing C sequence handling functions.

Functions

- `std::string` [convert_to_std_string](#) (void *str)
Convert a `rosidl_runtime_c__String` into `std::string`.
- `std::vector< std::string >` [convert_to_std_vector_string](#) (void *str_array, size_t size)
Convert a `rosidl_runtime_c__String` into a vector of `std::string`.
- `std::vector< std::string >` [convert_sequence_to_std_vector_string](#) (void *str_seq)
Convert a `rosidl_runtime_c__String__Sequence` into a vector of `std::string`.
- `std::u16string` [convert_to_std_u16string](#) (void *str)
Convert a `rosidl_runtime_c__U16String` into `std::u16string`.
- `std::vector< std::u16string >` [convert_to_std_vector_u16string](#) (void *str_array, size_t size)
Convert a `rosidl_runtime_c__U16String` into a vector of `std::u16string`.
- `std::vector< std::u16string >` [convert_sequence_to_std_vector_u16string](#) (void *str_seq)
Convert a `rosidl_runtime_c__U16String__Sequence` into a vector of `std::u16string`.
- void [assign_string](#) (std::string str, void *field)
Assing to a `rosidl_runtime_c__String` the value contained in a `std::string`.
- void [assign_vector_string](#) (std::vector< std::string > cpp_string_vector, void *str_array, size_t size)
Assing to a `rosidl_runtime_c__String` the value contained in a vector of `std::string`.
- void [assign_vector_string_to_sequence](#) (std::vector< std::string > cpp_string_vector, void *str_seq)
Assing to a `rosidl_runtime_c__String__Sequence` the value contained in a vector of `std::string`.
- void [assign_u16string](#) (std::u16string str, void *field)
Assing to a `rosidl_runtime_c__U16String` the value contained in a `std::u16string`.
- void [assign_vector_u16string](#) (std::vector< std::u16string > cpp_string_vector, void *str_array, size_t size)
Assing to a `rosidl_runtime_c__U16String` the value contained in a vector of `std::u16string`.
- void [assign_vector_u16string_to_sequence](#) (std::vector< std::u16string > cpp_string_vector, void *str_seq)
Assing to a `rosidl_runtime_c__U16String__Sequence` the value contained in a vector of `std::u16string`.

5.1.1 Detailed Description

Namespace containing C sequence handling functions.

The C data type implementation is more complicated than the C++ one, because complex types like vectors have to be manually managed and this header contains functions to convert C strings and generic sequences into respectively C++ strings and vectors.

5.1.2 Function Documentation

5.1.2.1 `assign_string()`

```
void CStringHelper::assign_string (
    std::string str,
    void * field)
```

Assing to a `rosidl_runtime_c__String` the value contained in a `std::string`.

This function stores the data contained in a C++ string in a `rosidl_runtime_c__String` pointed by the `field` parameter.

Parameters

<i>str</i>	C++ style string containing data
<i>field</i>	Pointer containing the destination of the string

5.1.2.2 `assign_u16string()`

```
void CStringHelper::assign_u16string (
    std::u16string str,
    void * field)
```

Assing to a `rosidl_runtime_c__U16String` the value contained in a `std::u16string`.

This function stores the data contained in a C++ `u16string` in a `rosidl_runtime_c__U16String` pointed by the `field` parameter.

Parameters

<i>str</i>	C++ style <code>u16string</code> containing data
<i>field</i>	Pointer containing the destination of the <code>u16string</code>

5.1.2.3 assign_vector_string()

```
void CStringHelper::assign_vector_string (
    std::vector< std::string > cpp_string_vector,
    void * str_array,
    size_t size)
```

Assigning to a `rosidl_runtime_c__String` the value contained in a vector of `std::string`.

This function stores the data contained in a C++ vector of strings in a `rosidl_runtime_c__String` fixed size sequence pointed by the `str_array` parameter.

Parameters

<i>cpp_string_vector</i>	C++ style vector of string containing data
<i>str_array</i>	Pointer containing the destination of the string sequence
<i>size</i>	Number of elements in the array

5.1.2.4 assign_vector_string_to_sequence()

```
void CStringHelper::assign_vector_string_to_sequence (
    std::vector< std::string > cpp_string_vector,
    void * str_seq)
```

Assigning to a `rosidl_runtime_c__String__Sequence` the value contained in a vector of `std::string`.

This function stores the data contained in a C++ vector of strings in a `rosidl_runtime_c__String__Sequence` variable size sequence pointed by the `str_array` parameter.

Parameters

<i>cpp_string_vector</i>	C++ style vector of string containing data
<i>str_seq</i>	Pointer containing the destination of the string sequence

5.1.2.5 assign_vector_u16string()

```
void CStringHelper::assign_vector_u16string (
    std::vector< std::u16string > cpp_string_vector,
    void * str_array,
    size_t size)
```

Assigning to a `rosidl_runtime_c__U16String` the value contained in a vector of `std::u16string`.

This function stores the data contained in a C++ vector of u16strings in a `rosidl_runtime_c__U16String` fixed size sequence pointed by the `str_array` parameter.

Parameters

<i>cpp_string_vector</i>	C++ style vector of u16strings containing data
<i>str_array</i>	Pointer containing the destination of the u16string sequence
<i>size</i>	Number of elements in the array

5.1.2.6 assign_vector_u16string_to_sequence()

```
void CStringHelper::assign_vector_u16string_to_sequence (
    std::vector< std::u16string > cpp_string_vector,
    void * str_seq)
```

Assigning to a `rosidl_runtime_c__U16String__Sequence` the value contained in a vector of `std::u16string`.

This function stores the data contained in a C++ vector of u16strings in a `rosidl_runtime_c__U16String__Sequence` variable size sequence pointed by the `str_array` parameter.

Parameters

<i>cpp_string_vector</i>	C++ style vector of u16strings containing data
<i>str_seq</i>	Pointer containing the destination of the u16string sequence

5.1.2.7 convert_sequence_to_std_vector_string()

```
std::vector< std::string > CStringHelper::convert_sequence_to_std_vector_string (
    void * str_seq)
```

Convert a `rosidl_runtime_c__String__Sequence` into a vector of `std::string`.

This function converts a `rosidl_runtime_c__String__Sequence` variable size sequence into a C++ vector of strings.

Parameters

<i>str_seq</i>	Pointer to the first original C-style string
----------------	--

Returns

A C++ vector of strings

5.1.2.8 convert_sequence_to_std_vector_u16string()

```
std::vector< std::u16string > CStringHelper::convert_sequence_to_std_vector_u16string (
    void * str_seq)
```

Convert a `rosidl_runtime_c__U16String__Sequence` into a vector of `std::u16string`.

This function converts a `rosidl_runtime_c__U16String__Sequence` variable size sequence into a C++ vector of `u16string`.

Parameters

<i>str_seq</i>	Pointer to the first original C-style u16string
----------------	---

Returns

A C++ vector of `u16string`

5.1.2.9 convert_to_std_string()

```
std::string CStringHelper::convert_to_std_string (
    void * str)
```

Convert a `rosidl_runtime_c__String` into `std::string`.

This function converts a `rosidl_runtime_c__String` into a C++ string.

Parameters

<i>str</i>	The original C-style string
------------	-----------------------------

Returns

A C++ string

5.1.2.10 convert_to_std_u16string()

```
std::u16string CStringHelper::convert_to_std_u16string (
    void * str)
```

Convert a `rosidl_runtime_c__U16String` into `std::u16string`.

This function converts a `rosidl_runtime_c__U16String` into a C++ `u16string`.

Parameters

<i>str</i>	The original C-style u16string
------------	--------------------------------

Returns

A C++ `u16string`

5.1.2.11 `convert_to_std_vector_string()`

```
std::vector< std::string > CStringHelper::convert_to_std_vector_string (
    void * str_array,
    size_t size)
```

Convert a `rosidl_runtime_c__String` into a vector of `std::string`.

This function converts a `rosidl_runtime_c__String` fixed size sequence into a C++ vector of strings.

Parameters

<i>str_array</i>	Pointer to the first original C-style string
<i>size</i>	Number of elements in the array

Returns

A C++ vector of strings

5.1.2.12 `convert_to_std_vector_u16string()`

```
std::vector< std::u16string > CStringHelper::convert_to_std_vector_u16string (
    void * str_array,
    size_t size)
```

Convert a `rosidl_runtime_c__U16String` into a vector of `std::u16string`.

This function converts a `rosidl_runtime_c__U16String` fixed size sequence into a C++ vector of `u16string`.

Parameters

<i>str_array</i>	Pointer to the first original C-style <code>u16string</code>
<i>size</i>	Number of elements in the array

Returns

A C++ vector of `u16strings`

5.2 Discovery Namespace Reference

Namespace containing discovery functions.

Typedefs

- using **DemangleFunction** = `std::string (*)(const std::string &)`

Functions

- char * **integer_to_string** (int x)
- std::string **resolve_prefix** (const std::string &name, const std::string &prefix)
Resolve a prefix.
- std::string **demangle_publisher_from_topic** (const std::string &topic_name)
Demangle a publisher.
- std::string **demangle_subscriber_from_topic** (const std::string &topic_name)
Demangle a subscriber.
- std::string **demangle_topic** (const std::string &topic_name)
Demangle a topic.
- std::string **demangle_service_request_from_topic** (const std::string &topic_name)
Demangle a service request.
- std::string **demangle_service_reply_from_topic** (const std::string &topic_name)
Demangle a service reply.
- std::string **demangle_service_from_topic** (const std::string &topic_name)
Demangle a service.
- std::string **identity_demangle** (const std::string &name)
No demangle.
- void **discovery_thread** (rmw_context_impl_t *impl)
Thread handling discovery beacons.
- rmw_ret_t **discovery_thread_start** (rmw_context_impl_t *impl)
Initialize the discovery thread.
- rmw_ret_t **discovery_thread_stop** (rmw_context_impl_t *impl)
Stop the discovery thread.
- void **send_discovery_beacon** (cbor::TxStream stream, std::string node_name, std::string node_namespace, int entity_type, rmw_gid_t entity_gid, std::string topic_name, std::string type_name, bool disconnect)
Send a discovery beacon.
- void **send_discovery_request** (cbor::TxStream stream)
Send a discovery request.

Variables

- const char *const **ros_topic_publisher_prefix** = integer_to_string(PUBLISHER_TYPE)
- const char *const **ros_topic_subscriber_prefix** = integer_to_string(SUBSCRIBER_TYPE)
- const char *const **ros_service_requester_prefix** = integer_to_string(CLIENT_TYPE)
- const char *const **ros_service_response_prefix** = integer_to_string(SERVICE_TYPE)

5.2.1 Detailed Description

Namespace containing discovery functions.

The middleware layer of a ROS stack must implement functionalities used to inform each node of the network structure of the other nodes connected, with their names and topics. Since this operation is quite resource-consuming and the underwater channel has a limited bandwidth, it is possible to disable it.

5.2.2 Function Documentation

5.2.2.1 demangle_publisher_from_topic()

```
std::string Discovery::demangle_publisher_from_topic (  
    const std::string & topic_name)
```

Demangle a publisher.

Return the topic name for a given topic if it is part of a publisher, else "".

Parameters

<i>topic_name</i>	Mangled topic name
-------------------	--------------------

Returns

Demangled topic name

5.2.2.2 demangle_service_from_topic()

```
std::string Discovery::demangle_service_from_topic (  
    const std::string & topic_name)
```

Demangle a service.

Return the service name for a given topic if it is part of a service, else "".

Parameters

<i>topic_name</i>	Mangled topic name
-------------------	--------------------

Returns

Demangled topic name

5.2.2.3 demangle_service_reply_from_topic()

```
std::string Discovery::demangle_service_reply_from_topic (  
    const std::string & topic_name)
```

Demangle a service reply.

Return the service name for a given topic if it is part of a service reply, else "".

Parameters

<i>topic_name</i>	Mangled topic name
-------------------	--------------------

Returns

Demangled topic name

5.2.2.4 demangle_service_request_from_topic()

```
std::string Discovery::demangle_service_request_from_topic (  
    const std::string & topic_name)
```

Demangle a service request.

Return the service name for a given topic if it is part of a service request, else "".

Parameters

<i>topic_name</i>	Mangled topic name
-------------------	--------------------

Returns

Demangled topic name

5.2.2.5 demangle_subscriber_from_topic()

```
std::string Discovery::demangle_subscriber_from_topic (  
    const std::string & topic_name)
```

Demangle a subscriber.

Return the topic name for a given topic if it is part of a subscriber, else "".

Parameters

<i>topic_name</i>	Mangled topic name
-------------------	--------------------

Returns

Demangled topic name

5.2.2.6 demangle_topic()

```
std::string Discovery::demangle_topic (
    const std::string & topic_name)
```

Demangle a topic.

Return the topic name for a given topic if it is part of one, else "".

Parameters

<i>topic_name</i>	Mangled topic name
-------------------	--------------------

Returns

Demangled topic name

5.2.2.7 discovery_thread()

```
void Discovery::discovery_thread (
    rmw_context_impl_t * impl)
```

Thread handling discovery beacons.

This function allows the middleware to receive and process incoming discovery beacons from the other nodes.

Parameters

<i>impl</i>	The middleware context implementation
-------------	---------------------------------------

5.2.2.8 discovery_thread_start()

```
rmw_ret_t Discovery::discovery_thread_start (
    rmw_context_impl_t * impl)
```

Initialize the discovery thread.

This function is called during the initialization of the middleware and starts the discovery thread.

Parameters

<i>impl</i>	The middleware context implementation
-------------	---------------------------------------

Returns

Outcome of the operation

5.2.2.9 discovery_thread_stop()

```
rmw_ret_t Discovery::discovery_thread_stop (  
    rmw_context_impl_t * impl)
```

Stop the discovery thread.

This function is called during the termination of the middleware and stops the discovery thread.

Parameters

<i>impl</i>	The middleware context implementation
-------------	---------------------------------------

Returns

Outcome of the operation

5.2.2.10 identity_demangle()

```
std::string Discovery::identity_demangle (  
    const std::string & name)
```

No demangle.

Used when ros names are not mangled.

Parameters

<i>name</i>	Topic name
-------------	------------

Returns

Same topic name

5.2.2.11 resolve_prefix()

```
std::string Discovery::resolve_prefix (  
    const std::string & name,  
    const std::string & prefix)
```

Resolve a prefix.

Returns `name` stripped of `prefix`.

Parameters

<i>name</i>	Mangled topic name
-------------	--------------------

<i>prefix</i>	Prefix of the entity type
---------------	---------------------------

Returns

Demangled topic name

5.2.2.12 send_discovery_beacon()

```
void Discovery::send_discovery_beacon (
    cbor::TxStream stream,
    std::string node_name,
    std::string node_namespace,
    int entity_type,
    rmw_gid_t entity_gid,
    std::string topic_name,
    std::string type_name,
    bool disconnect)
```

Send a discovery beacon.

This function sends a beacon in the underwater channel containing all the informations related to a specific entity of a node.

Parameters

<i>stream</i>	The stream used to send data
<i>node_name</i>	The name of the node holding the entity
<i>node_namespace</i>	The namespace of the node holding the entity
<i>entity_type</i>	The type of the entity
<i>entity_gid</i>	The global identifier of the entity
<i>topic_name</i>	The topic name
<i>type_name</i>	The topic type
<i>disconnect</i>	Flag used to determine if an entity is connecting or disconnecting

5.2.2.13 send_discovery_request()

```
void Discovery::send_discovery_request (
    cbor::TxStream stream)
```

Send a discovery request.

This function sends a request in the underwater channel to all the nodes requiring them to send their discovery beacons.

Parameters

<code>stream</code>	The stream used to send data
---------------------	------------------------------

5.3 MessageSerialization Namespace Reference

Namespace containing serialization functions.

Functions

- `template<typename T>`
`void serialize_field (const INTROSPECTION_CPP_MEMBER *member, void *field, cbor::TxStream &stream)`
Serialize a C++ field.
- `template<typename T>`
`void serialize_field (const INTROSPECTION_C_MEMBER *member, void *field, cbor::TxStream &stream)`
Serialize a C field.
- `template<typename MembersType>`
`void serialize (const void *msg, const MembersType *casted_members, cbor::TxStream &stream)`
Serialize a ROS message, request or response.
- `template<typename T>`
`void deserialize_field (const INTROSPECTION_CPP_MEMBER *member, void *field, cbor::RxStream &stream)`
Deserialize a C++ field.
- `template<typename T>`
`void deserialize_field (const INTROSPECTION_C_MEMBER *member, void *field, cbor::RxStream &stream)`
Deserialize a C field.
- `template<typename MembersType>`
`void deserialize (void *msg, const MembersType *casted_members, cbor::RxStream &stream)`
Deserialize a ROS message, request or response.

5.3.1 Detailed Description

Namespace containing serialization functions.

The message data structure coming from upper layers is interpreted using type support informations passed by ROS2 during the creation of publishers, subscribers, clients and services. Those functions are used to compute the exact position that every data type must assume in memory and then calls TxStream or RxStream to receive or write them in the assigned location.

5.3.2 Function Documentation

5.3.2.1 deserialize()

```
template<typename MembersType>
void MessageSerialization::deserialize (
    void * msg,
    const MembersType * casted_members,
    cbor::RxStream & stream)
```

Deserialize a ROS message, request or response.

Every time DESERT receives data from the channel a memory location is used to store the corresponding member type, and this function merges all the elementary C or C++ types into the whole message. To perform this operation the `deserialize_field` function is called to decode every specific data.

Parameters

<i>msg</i>	Pointer to the first byte of the message in memory
<i>casted_members</i>	Pointer to the member containing type support informations
<i>stream</i>	The stream used to receive data

5.3.2.2 `deserialize_field()` [1/2]

```
template<typename T>
void MessageSerialization::deserialize_field (
    const INTROSPECTION_C_MEMBER * member,
    void * field,
    cbor::RxStream & stream)
```

Deserialize a C field.

The type support introspection information is used to know if a specific data type is a single item, a sequence or a variable length sequence. Based on this conclusion a specific interpretation is passed to the stream.

Parameters

<i>member</i>	Pointer to the member containing type support informations
<i>field</i>	Pointer to the destination memory address of the elementary data
<i>stream</i>	The stream used to receive data

5.3.2.3 `deserialize_field()` [2/2]

```
template<typename T>
void MessageSerialization::deserialize_field (
    const INTROSPECTION_CPP_MEMBER * member,
    void * field,
    cbor::RxStream & stream)
```

Deserialize a C++ field.

The type support introspection information is used to know if a specific data type is a single item, a sequence or a vector. Based on this conclusion a specific interpretation is passed to the stream.

Parameters

<i>member</i>	Pointer to the member containing type support informations
<i>field</i>	Pointer to the destination memory address of the elementary data

<i>stream</i>	The stream used to receive data
---------------	---------------------------------

5.3.2.4 serialize()

```
template<typename MembersType>
void MessageSerialization::serialize (
    const void * msg,
    const MembersType * casted_members,
    cbor::TxStream & stream)
```

Serialize a ROS message, request or response.

Every time ROS has data to send in the channel a memory location is passed with the corresponding message member type, and this function separates all the fields into elementary C or C++ types. Then the `serialize_field` function is called to encode the specific data.

Parameters

<i>msg</i>	Pointer to the first byte of the message in memory
<i>casted_members</i>	Pointer to the member containing type support informations
<i>stream</i>	The stream used to send data

5.3.2.5 serialize_field() [1/2]

```
template<typename T>
void MessageSerialization::serialize_field (
    const INTROSPECTION_C_MEMBER * member,
    void * field,
    cbor::TxStream & stream)
```

Serialize a C field.

The type support introspection information is used to know if a specific data type is a single item, a sequence or a variable length sequence. Based on this conclusion a specific interpretation is passed to the stream.

Parameters

<i>member</i>	Pointer to the member containing type support informations
<i>field</i>	Pointer to the origin memory address of the elementary data
<i>stream</i>	The stream used to send data

5.3.2.6 `serialize_field()` [2/2]

```
template<typename T>
void MessageSerialization::serialize_field (
    const INTROSPECTION_CPP_MEMBER * member,
    void * field,
    cbor::TxStream & stream)
```

Serialize a C++ field.

The type support introspection information is used to know if a specific data type is a single item, a sequence or a vector. Based on this conclusion a specific interpretation is passed to the stream.

Parameters

<i>member</i>	Pointer to the member containing type support informations
<i>field</i>	Pointer to the origin memory address of the elementary data
<i>stream</i>	The stream used to send data

Chapter 6

Class Documentation

6.1 security::AeadParams Class Reference

Public Member Functions

- [AeadParams](#) (AeadAlgorithms alg)
Creates the parameter set for an AEAD algorithm.
- AeadAlgorithms [get_alg](#) () const
Returns the configured AEAD algorithm.
- uint16_t [get_key_size](#) () const
Returns the size of the AEAD key.
- uint16_t [get_nonce_size](#) () const
Returns the size of the AEAD nonce.
- uint16_t [get_tag_size](#) () const
Returns the size of the AEAD authentication tag.

6.1.1 Constructor & Destructor Documentation

6.1.1.1 AeadParams()

```
security::AeadParams::AeadParams (  
    AeadAlgorithms alg)
```

Creates the parameter set for an AEAD algorithm.

Parameters

<i>alg</i>	AEAD algorithm to configure.
------------	------------------------------

6.1.2 Member Function Documentation

6.1.2.1 get_alg()

```
AeadAlgorithms security::AeadParams::get_alg () const [inline]
```

Returns the configured AEAD algorithm.

Returns

The selected AEAD algorithm identifier.

6.1.2.2 get_key_size()

```
uint16_t security::AeadParams::get_key_size () const [inline]
```

Returns the size of the AEAD key.

Returns

Key size in bytes.

6.1.2.3 get_nonce_size()

```
uint16_t security::AeadParams::get_nonce_size () const [inline]
```

Returns the size of the AEAD nonce.

Returns

Nonce size in bytes.

6.1.2.4 get_tag_size()

```
uint16_t security::AeadParams::get_tag_size () const [inline]
```

Returns the size of the AEAD authentication tag.

Returns

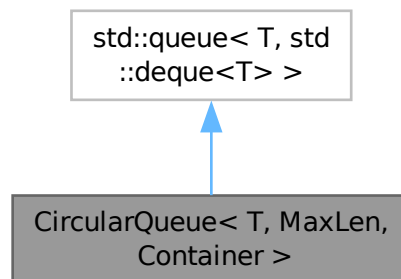
Authentication-tag size in bytes.

The documentation for this class was generated from the following files:

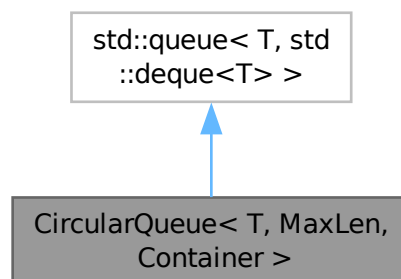
- src/desert_classes/security/[AeadParams.h](#)
- src/desert_classes/security/AeadParams.cpp

6.2 CircularQueue< T, MaxLen, Container > Class Template Reference

Inheritance diagram for CircularQueue< T, MaxLen, Container >:



Collaboration diagram for CircularQueue< T, MaxLen, Container >:



Public Member Functions

- void **push** (const T &value)

The documentation for this class was generated from the following file:

- src/desert_classes/[CBorStream.h](#)

6.3 DesertClient Class Reference

Public Member Functions

- [DesertClient](#) (std::string service_name, const rosidl_service_type_support_t *type_supports, rmw_gid_t gid)
Create a client.
- bool [has_data](#) ()
Check if there is available data for the current client instance.
- void [send_request](#) (const void *req, int64_t *sequence_id)
Send a request to the service.
- void [read_response](#) (void *res, rmw_service_info_t *req_header)
Read a response from the service.
- rmw_gid_t [get_gid](#) ()
Retreive the gid of the current entity.
- std::string [get_service_name](#) ()
Retreive the service name of the current entity.
- std::string [get_request_type_name](#) ()
Retreive the request type of the current entity.
- std::string [get_response_type_name](#) ()
Retreive the response type of the current entity.

6.3.1 Constructor & Destructor Documentation

6.3.1.1 DesertClient()

```
DesertClient::DesertClient (
    std::string service_name,
    const rosidl_service_type_support_t * type_supports,
    rmw_gid_t gid)
```

Create a client.

Parameters

<i>service_name</i>	Name of the service to send requests and receive responses
<i>type_supports</i>	Pointer to the message data structure coming from the ROS upper layers
<i>gid</i>	Global identifier of the entity

6.3.2 Member Function Documentation

6.3.2.1 get_gid()

```
rmw_gid_t DesertClient::get_gid ()
```

Retreive the gid of the current entity.

This function returns the global identifier of the current entity in the rmw format.

Returns

Global identifier of the entity

6.3.2.2 get_request_type_name()

```
std::string DesertClient::get_request_type_name ()
```

Retreive the request type of the current entity.

This function returns a string containing the service request type name of the current entity.

Returns

Type of the service request

6.3.2.3 get_response_type_name()

```
std::string DesertClient::get_response_type_name ()
```

Retreive the response type of the current entity.

This function returns a string containing the service response type name of the current entity.

Returns

Type of the service response

6.3.2.4 get_service_name()

```
std::string DesertClient::get_service_name ()
```

Retreive the service name of the current entity.

This function returns a string containing the service name of the current entity.

Returns

Name of the service

6.3.2.5 has_data()

```
bool DesertClient::has_data ()
```

Check if there is available data for the current client instance.

The has_data function calls the interpret_packets method in RxStream and then verifies if in the map of client packets there is a correspondence with the service name and the sequence identifier of the current instance.

Returns

True if data is present otherwise false

6.3.2.6 read_response()

```
void DesertClient::read_response (
    void * res,
    rmw_service_info_t * req_header)
```

Read a response from the service.

The read_response function interprets a transmission with the current sequence identifier deserializing the message using the method from the [MessageSerialization](#) namespace. A discrimination is made between C members and C++ members based on the type support.

Parameters

<i>res</i>	Pointer to the memory location used to store the reading
<i>req_header</i>	Pointer to the request header used to store the service sequence identifier

6.3.2.7 send_request()

```
void DesertClient::send_request (
    const void * req,
    int64_t * sequence_id)
```

Send a request to the service.

The send_request function starts a transmission with the current sequence identifier and then serializes the message using the method from the [MessageSerialization](#) namespace. A discrimination is made between C members and C++ members based on the type support.

Parameters

<i>req</i>	Pointer to the request to send
<i>sequence_id</i>	Pointer to the random service sequence identifier

The documentation for this class was generated from the following files:

- src/desert_classes/[DesertClient.h](#)
- src/desert_classes/DesertClient.cpp

6.4 DesertGuardCondition Class Reference

Public Member Functions

- **DesertGuardCondition ()**
Create a guard condition.
- void [trigger \(\)](#)
Trigger the guard condition.
- bool [has_triggered \(\)](#)
Check if the guard condition has triggered.
- bool [get_has_triggered \(\)](#)
Check if the guard condition has triggered.

6.4.1 Member Function Documentation

6.4.1.1 get_has_triggered()

```
bool DesertGuardCondition::get_has_triggered ()
```

Check if the guard condition has triggered.

The `get_has_triggered` function returns a `bool` value with the status of the guard condition and resets the internal state to `false`.

Returns

True if the guard condition has triggered otherwise `false`

6.4.1.2 has_triggered()

```
bool DesertGuardCondition::has_triggered ()
```

Check if the guard condition has triggered.

The `has_triggered` function returns a `bool` value with the status of the guard condition. Its internal state is not modified.

Returns

True if the guard condition has triggered otherwise `false`

6.4.1.3 trigger()

```
void DesertGuardCondition::trigger ()
```

Trigger the guard condition.

The `trigger` function sets the status of the atomic `bool` variable `_has_triggered` to save the new status of the guard condition.

The documentation for this class was generated from the following files:

- `src/desert_classes/DesertGuardCondition.h`
- `src/desert_classes/DesertGuardCondition.cpp`

6.5 DesertNode Class Reference

Public Member Functions

- [DesertNode](#) (std::string name, std::string namespace_, rmw_gid_t gid)
Create a node.
- void [add_publisher](#) ([DesertPublisher](#) *pub)
Add a publisher to the current node.
- void [add_subscriber](#) ([DesertSubscriber](#) *sub)
Add a subscriber to the current node.
- void [add_client](#) ([DesertClient](#) *cli)
Add a client to the current node.
- void [add_service](#) ([DesertService](#) *ser)
Add a service to the current node.
- void [remove_publisher](#) ([DesertPublisher](#) *pub)
Remove a publisher from the current node.
- void [remove_subscriber](#) ([DesertSubscriber](#) *sub)
Remove a subscriber from the current node.
- void [remove_client](#) ([DesertClient](#) *cli)
Remove a client from the current node.
- void [remove_service](#) ([DesertService](#) *ser)
Remove a service from the current node.
- rmw_gid_t [get_gid](#) ()
Retreive the gid of the current entity.

6.5.1 Constructor & Destructor Documentation

6.5.1.1 DesertNode()

```
DesertNode::DesertNode (
    std::string name,
    std::string namespace_,
    rmw_gid_t gid)
```

Create a node.

Parameters

<i>name</i>	Name of the node
<i>namespace_</i>	Namespace of the node
<i>gid</i>	Global identifier of the node

6.5.2 Member Function Documentation

6.5.2.1 add_client()

```
void DesertNode::add_client (  
    DesertClient * cli)
```

Add a client to the current node.

This function pushes the pointer to a client in a vector of all the registered clients related to the current node.

Parameters

<i>cli</i>	Pointer to a DesertClient instance
------------	--

6.5.2.2 add_publisher()

```
void DesertNode::add_publisher (  
    DesertPublisher * pub)
```

Add a publisher to the current node.

This function pushes the pointer to a publisher in a vector of all the registered publishers related to the current node.

Parameters

<i>pub</i>	Pointer to a DesertPublisher instance
------------	---

6.5.2.3 add_service()

```
void DesertNode::add_service (  
    DesertService * ser)
```

Add a service to the current node.

This function pushes the pointer to a service in a vector of all the registered services related to the current node.

Parameters

<i>ser</i>	Pointer to a DesertService instance
------------	---

6.5.2.4 add_subscriber()

```
void DesertNode::add_subscriber (
    DesertSubscriber * sub)
```

Add a subscriber to the current node.

This function pushes the pointer to a subscriber in a vector of all the registered subscribers related to the current node.

Parameters

<i>sub</i>	Pointer to a DesertSubscriber instance
------------	--

6.5.2.5 get_gid()

```
rmw_gid_t DesertNode::get_gid ()
```

Retrieve the gid of the current entity.

This function returns the global identifier of the current entity in the rmw format.

Returns

Global identifier of the entity

6.5.2.6 remove_client()

```
void DesertNode::remove_client (
    DesertClient * cli)
```

Remove a client from the current node.

This function removes the pointer to a client from the vector of all the registered clients related to the current node.

Parameters

<i>cli</i>	Pointer to a DesertClient instance
------------	--

6.5.2.7 remove_publisher()

```
void DesertNode::remove_publisher (
    DesertPublisher * pub)
```

Remove a publisher from the current node.

This function removes the pointer to a publisher from the vector of all the registered publishers related to the current node.

Parameters

<i>pub</i>	Pointer to a DesertPublisher instance
------------	---

6.5.2.8 remove_service()

```
void DesertNode::remove_service (
    DesertService * ser)
```

Remove a service from the current node.

This function removes the pointer to a service from the vector of all the registered services related to the current node.

Parameters

<i>ser</i>	Pointer to a DesertService instance
------------	---

6.5.2.9 remove_subscriber()

```
void DesertNode::remove_subscriber (
    DesertSubscriber * sub)
```

Remove a subscriber from the current node.

This function removes the pointer to a subscriber from the vector of all the registered subscribers related to the current node.

Parameters

<i>sub</i>	Pointer to a DesertSubscriber instance
------------	--

The documentation for this class was generated from the following files:

- [src/desert_classes/DesertNode.h](#)
- [src/desert_classes/DesertNode.cpp](#)

6.6 DesertPublisher Class Reference

Public Member Functions

- [DesertPublisher](#) (std::string topic_name, const rosidl_message_type_support_t *type_supports, rmw_gid_t gid)
Create a publisher.
- void [push](#) (const void *msg)
Send a publication on the topic.
- rmw_gid_t [get_gid](#) ()
Retrieve the gid of the current entity.
- std::string [get_topic_name](#) ()
Retrieve the topic name of the current entity.
- std::string [get_type_name](#) ()
Retrieve the message type of the current entity.

6.6.1 Constructor & Destructor Documentation

6.6.1.1 DesertPublisher()

```
DesertPublisher::DesertPublisher (
    std::string topic_name,
    const rosidl_message_type_support_t * type_supports,
    rmw_gid_t gid)
```

Create a publisher.

Parameters

<i>topic_name</i>	Name of the topic used to push the messages
<i>type_supports</i>	Pointer to the message data structure coming from the ROS upper layers
<i>gid</i>	Global identifier of the entity

6.6.2 Member Function Documentation

6.6.2.1 get_gid()

```
rmw_gid_t DesertPublisher::get_gid ()
```

Retrieve the gid of the current entity.

This function returns the global identifier of the current entity in the rmw format.

Returns

Global identifier of the entity

6.6.2.2 get_topic_name()

```
std::string DesertPublisher::get_topic_name ()
```

Retrieve the topic name of the current entity.

This function returns a string containing the topic name of the current entity.

Returns

Name of the topic

6.6.2.3 get_type_name()

```
std::string DesertPublisher::get_type_name ()
```

Retrieve the message type of the current entity.

This function returns a string containing the message type name of the current entity.

Returns

Type of the message

6.6.2.4 push()

```
void DesertPublisher::push (  
    const void * msg)
```

Send a publication on the topic.

The push function starts a transmission with the topic name in the current instance and then serializes the message using the method from the [MessageSerialization](#) namespace. A discrimination is made between C members and C++ members based on the type support.

Parameters

<i>msg</i>	Pointer to the message to send
------------	--------------------------------

The documentation for this class was generated from the following files:

- [src/desert_classes/DesertPublisher.h](#)
- [src/desert_classes/DesertPublisher.cpp](#)

6.7 DesertService Class Reference

Public Member Functions

- [DesertService](#) (std::string service_name, const rosidl_service_type_support_t *type_supports, rmw_gid_t gid)
Create a service.
- bool [has_data](#) ()
Check if there is available data for the current service instance.
- void [read_request](#) (void *req, rmw_service_info_t *req_header)
Read a request from a client.
- void [send_response](#) (void *res, rmw_request_id_t *req_header)
Send the response to a client.
- rmw_gid_t [get_gid](#) ()
Retrieve the gid of the current entity.
- std::string [get_service_name](#) ()
Retrieve the service name of the current entity.
- std::string [get_request_type_name](#) ()
Retrieve the request type of the current entity.
- std::string [get_response_type_name](#) ()
Retrieve the response type of the current entity.

6.7.1 Constructor & Destructor Documentation

6.7.1.1 DesertService()

```
DesertService::DesertService (
    std::string service_name,
    const rosidl_service_type_support_t * type_supports,
    rmw_gid_t gid)
```

Create a service.

Parameters

<i>service_name</i>	Name of the service to receive requests and send responses
<i>type_supports</i>	Pointer to the message data structure coming from the ROS upper layers
<i>gid</i>	Global identifier of the entity

6.7.2 Member Function Documentation

6.7.2.1 get_gid()

```
rmw_gid_t DesertService::get_gid ()
```

Retrieve the gid of the current entity.

This function returns the global identifier of the current entity in the rmw format.

Returns

Global identifier of the entity

6.7.2.2 get_request_type_name()

```
std::string DesertService::get_request_type_name ()
```

Retreive the request type of the current entity.

This function returns a string containing the service request type name of the current entity.

Returns

Type of the service request

6.7.2.3 get_response_type_name()

```
std::string DesertService::get_response_type_name ()
```

Retreive the response type of the current entity.

This function returns a string containing the service response type name of the current entity.

Returns

Type of the service response

6.7.2.4 get_service_name()

```
std::string DesertService::get_service_name ()
```

Retreive the service name of the current entity.

This function returns a string containing the service name of the current entity.

Returns

Name of the service

6.7.2.5 has_data()

```
bool DesertService::has_data ()
```

Check if there is available data for the current service instance.

The has_data function calls the interpret_packets method in RxStream and then verifies if in the map of service packets there is a correspondence with the service name of the current instance.

Returns

True if data is present otherwise false

6.7.2.6 read_request()

```
void DesertService::read_request (
    void * req,
    rmw_service_info_t * req_header)
```

Read a request from a client.

The read_request function interprets a transmission with the service name in the current instance deserializing the message using the method from the [MessageSerialization](#) namespace. A discrimination is made between C members and C++ members based on the type support.

Parameters

<i>req</i>	Pointer to the memory location used to store the request
<i>req_header</i>	Pointer to the request header used to store the service sequence identifier

6.7.2.7 send_response()

```
void DesertService::send_response (
    void * res,
    rmw_request_id_t * req_header)
```

Send the response to a client.

The send_response function starts a transmission with the sequence identifier in req_header and then serializes the message using the method from the [MessageSerialization](#) namespace. A discrimination is made between C members and C++ members based on the type support.

Parameters

<i>res</i>	Pointer to the response to send
<i>req_header</i>	Pointer to the request header used to store the service sequence identifier

The documentation for this class was generated from the following files:

- src/desert_classes/[DesertService.h](#)
- src/desert_classes/DesertService.cpp

6.8 DesertSubscriber Class Reference

Public Member Functions

- [DesertSubscriber](#) (std::string topic_name, const rosidl_message_type_support_t *type_supports, rmw_gid_t gid)
Create a subscriber.
- bool [has_data](#) ()
Check if there is available data for the registered topic.
- void [read_data](#) (void *msg)

- *Read a publication from the publisher.*
`rmw_gid_t get_gid ()`
- *Retrieve the gid of the current entity.*
`std::string get_topic_name ()`
- *Retrieve the topic name of the current entity.*
`std::string get_type_name ()`
- *Retrieve the message type of the current entity.*

6.8.1 Constructor & Destructor Documentation

6.8.1.1 DesertSubscriber()

```
DesertSubscriber::DesertSubscriber (
    std::string topic_name,
    const rosidl_message_type_support_t * type_supports,
    rmw_gid_t gid)
```

Create a subscriber.

Parameters

<i>topic_name</i>	Name of the topic used for the registration
<i>type_supports</i>	Pointer to the message data structure coming from the ROS upper layers
<i>gid</i>	Global identifier of the entity

6.8.2 Member Function Documentation

6.8.2.1 get_gid()

```
rmw_gid_t DesertSubscriber::get_gid ()
```

Retrieve the gid of the current entity.

This function returns the global identifier of the current entity in the rmw format.

Returns

Global identifier of the entity

6.8.2.2 get_topic_name()

```
std::string DesertSubscriber::get_topic_name ()
```

Retrieve the topic name of the current entity.

This function returns a string containing the topic name of the current entity.

Returns

Name of the topic

6.8.2.3 get_type_name()

```
std::string DesertSubscriber::get_type_name ()
```

Retreive the message type of the current entity.

This function returns a string containing the message type name of the current entity.

Returns

Type of the message

6.8.2.4 has_data()

```
bool DesertSubscriber::has_data ()
```

Check if there is available data for the registered topic.

The has_data function calls the interpret_packets method in RxStream and then verifies if in the map of subscriber packets there is a correspondence with the topic name of the current instance.

Returns

True if data is present otherwise false

6.8.2.5 read_data()

```
void DesertSubscriber::read_data (  
    void * msg)
```

Read a publication from the publisher.

The read_data function interprets a transmission with the topic name present in the current instance deserializing the message using the method from the [MessageSerialization](#) namespace. A discrimination is made between C members and C++ members based on the type support.

Parameters

<i>msg</i>	Pointer to the memory location used to store the message
------------	--

The documentation for this class was generated from the following files:

- src/desert_classes/[DesertSubscriber.h](#)
- src/desert_classes/DesertSubscriber.cpp

6.9 DesertWaitset Class Reference

Public Attributes

- `std::mutex lock`
- `bool inuse`

The documentation for this class was generated from the following file:

- `src/desert_classes/DesertWaitSet.h`

6.10 cbor::TxStream::ERROR Struct Reference

Public Attributes

- `cbor_error_t libmcu_error`
- `nanocbor_error_t nanocbor_error`

The documentation for this struct was generated from the following files:

- `src/desert_classes/LibMcuCBorStream.cpp`
- `src/desert_classes/NanoCBorStream.cpp`

6.11 GenericCSequence< T > Struct Template Reference

The documentation for this struct was generated from the following file:

- `src/desert_classes/macros.h`

6.12 cbor::RxStream::ITEM Struct Reference

Public Attributes

- `cbor_item_t libmcu_item`
- `nanocbor_value_t nanocbor_value`

The documentation for this struct was generated from the following files:

- `src/desert_classes/LibMcuCBorStream.cpp`
- `src/desert_classes/NanoCBorStream.cpp`

6.13 rmw_context_impl_s Struct Reference

Public Attributes

- `rmw_dds_common::Context` **common**
- `bool` **is_shutdown** {false}

The documentation for this struct was generated from the following file:

- `src/desert_classes/rmw_context_impl_s.h`

6.14 cbor::RxStream Class Reference

Classes

- struct [ITEM](#)

Public Member Functions

- [RxStream](#) (uint8_t stream_type, std::string stream_name, uint8_t stream_identifier)
Create a reception stream.
- [~RxStream](#) ()
Destroy the reception stream.
- `bool` [data_available](#) (int64_t sequence_id=0)
Check if there are data.
- `void` [clear_buffer](#) ()
Clear the currently buffered packet.
- [RxStream](#) & [operator>>](#) (uint64_t &n)
Decode uint64.
- [RxStream](#) & [operator>>](#) (uint32_t &n)
Decode uint32.
- [RxStream](#) & [operator>>](#) (uint16_t &n)
Decode uint16.
- [RxStream](#) & [operator>>](#) (uint8_t &n)
Decode uint8.
- [RxStream](#) & [operator>>](#) (int64_t &n)
Decode int64.
- [RxStream](#) & [operator>>](#) (int32_t &n)
Decode int32.
- [RxStream](#) & [operator>>](#) (int16_t &n)
Decode int16.
- [RxStream](#) & [operator>>](#) (int8_t &n)
Decode int8.
- [RxStream](#) & [operator>>](#) (char &n)
Decode char.
- [RxStream](#) & [operator>>](#) (float &f)
Decode float.
- [RxStream](#) & [operator>>](#) (double &d)

- Decode double.*
- [RxStream & operator>>](#) (std::string &s)
- Decode string.*
- [RxStream & operator>>](#) (std::u16string &s)
- Decode u16string.*
- [RxStream & operator>>](#) (bool &b)
- Decode bool.*
- template<typename T>
[RxStream & operator>>](#) (std::vector< T > &v)
- Decode vector.*
- [RxStream & operator>>](#) (std::vector< bool > &v)
- Decode bool vector.*
- template<typename T>
[RxStream & deserialize_sequence](#) (T *items, size_t size)
- Deserialize a sequence of uniform elements.*
- uint8_t [get_type](#) () const
- Get the stream type of a specific instance.*
- std::string [get_name](#) () const
- Get the topic name of a specific instance.*
- uint8_t [get_identifier](#) () const
- Get the stream identifier of a specific instance.*
- void [push_packet](#) (std::vector< std::pair< void *, int > > packet)
- Add a packet to _received_packets.*

Static Public Member Functions

- static void [interpret_packets](#) ()
- Interpret raw packets and splits them into different communication types.*

6.14.1 Constructor & Destructor Documentation

6.14.1.1 RxStream()

```
cbor::RxStream::RxStream (
    uint8_t stream_type,
    std::string stream_name,
    uint8_t stream_identifier)
```

Create a reception stream.

Parameters

<i>stream_type</i>	Type of the object using the current instance
<i>stream_name</i>	Name of the topic or the service to which the communication belongs
<i>stream_identifier</i>	Identifier of the topic or the service read from configuration

6.14.2 Member Function Documentation

6.14.2.1 `clear_buffer()`

```
void cbor::RxStream::clear_buffer ()
```

Clear the currently buffered packet.

When the packet is read by the entity, this function must be called to clear the buffer and allow [RxStream](#) to add the next one in the queue.

6.14.2.2 `data_available()`

```
bool cbor::RxStream::data_available (
    int64_t sequence_id = 0)
```

Check if there are data.

A map contains the information received for all topics and services, so using the name saved in the current instance as key it is possible to know if a message is arrived for a specific entity.

Parameters

<i>sequence_id</i>	The id of the client service communication
--------------------	--

6.14.2.3 `deserialize_sequence()`

```
template<typename T>
RxStream & cbor::RxStream::deserialize_sequence (
    T * items,
    size_t size) [inline]
```

Deserialize a sequence of uniform elements.

Parameters

<i>items</i>	Pointer to the first element
<i>size</i>	Size of the items array

6.14.2.4 `get_identifier()`

```
uint8_t cbor::RxStream::get_identifier () const
```

Get the stream identifier of a specific instance.

Returns

Topic identifier of the stream

6.14.2.5 get_name()

```
std::string cbor::RxStream::get_name () const
```

Get the topic name of a specific instance.

Returns

Topic name of the stream

6.14.2.6 get_type()

```
uint8_t cbor::RxStream::get_type () const
```

Get the stream type of a specific instance.

Returns

Type of the stream

6.14.2.7 interpret_packets()

```
void cbor::RxStream::interpret_packets () [static]
```

Interpret raw packets and splits them into different communication types.

Raw packets from [TcpDaemon](#) are read and interpreted in order to put them in a map where the key allows to distinguish the topic name or the service name, and eventually the sequence identifier.

6.14.2.8 operator>>() [1/16]

```
RxStream & cbor::RxStream::operator>> (  
    bool & b)
```

Decode bool.

Parameters

<i>b</i>	Field to decode
----------	-----------------

6.14.2.9 `operator>>()` [2/16]

```
RxStream & cbor::RxStream::operator>> (  
    char & n)
```

Decode char.

Parameters

<i>n</i>	Field to decode
----------	-----------------

6.14.2.10 `operator>>()` [3/16]

```
RxStream & cbor::RxStream::operator>> (  
    double & d)
```

Decode double.

Parameters

<i>d</i>	Field to decode
----------	-----------------

6.14.2.11 `operator>>()` [4/16]

```
RxStream & cbor::RxStream::operator>> (  
    float & f)
```

Decode float.

Parameters

<i>f</i>	Field to decode
----------	-----------------

6.14.2.12 `operator>>()` [5/16]

```
RxStream & cbor::RxStream::operator>> (  
    int16_t & n)
```

Decode int16.

Parameters

<i>n</i>	Field to decode
----------	-----------------

6.14.2.13 operator>>() [6/16]

```
RxStream & cbor::RxStream::operator>> (
    int32_t & n)
```

Decode int32.

Parameters

<i>n</i>	Field to decode
----------	-----------------

6.14.2.14 operator>>() [7/16]

```
RxStream & cbor::RxStream::operator>> (
    int64_t & n)
```

Decode int64.

Parameters

<i>n</i>	Field to decode
----------	-----------------

6.14.2.15 operator>>() [8/16]

```
RxStream & cbor::RxStream::operator>> (
    int8_t & n)
```

Decode int8.

Parameters

<i>n</i>	Field to decode
----------	-----------------

6.14.2.16 operator>>() [9/16]

```
RxStream & cbor::RxStream::operator>> (
    std::string & s)
```

Decode string.

Parameters

<i>s</i>	Field to decode
----------	-----------------

6.14.2.17 operator>>() [10/16]

```
RxStream & cbor::RxStream::operator>> (
    std::u16string & s)
```

Decode u16string.

Parameters

<i>s</i>	Field to decode
----------	-----------------

6.14.2.18 operator>>() [11/16]

```
RxStream & cbor::RxStream::operator>> (
    std::vector< bool > & v)
```

Decode bool vector.

Parameters

<i>v</i>	Field to decode
----------	-----------------

6.14.2.19 operator>>() [12/16]

```
template<typename T>
RxStream & cbor::RxStream::operator>> (
    std::vector< T > & v) [inline]
```

Decode vector.

Parameters

<i>v</i>	Field to decode
----------	-----------------

6.14.2.20 operator>>() [13/16]

```
RxStream & cbor::RxStream::operator>> (
    uint16_t & n)
```

Decode uint16.

Parameters

<i>n</i>	Field to decode
----------	-----------------

6.14.2.21 operator>>() [14/16]

```
RxStream & cbor::RxStream::operator>> (
    uint32_t & n)
```

Decode uint32.

Parameters

<i>n</i>	Field to decode
----------	-----------------

6.14.2.22 operator>>() [15/16]

```
RxStream & cbor::RxStream::operator>> (
    uint64_t & n)
```

Decode uint64.

Parameters

<i>n</i>	Field to decode
----------	-----------------

6.14.2.23 operator>>() [16/16]

```
RxStream & cbor::RxStream::operator>> (
    uint8_t & n)
```

Decode uint8.

Parameters

<i>n</i>	Field to decode
----------	-----------------

6.14.2.24 push_packet()

```
void cbor::RxStream::push_packet (
    std::vector< std::pair< void *, int > > packet)
```

Add a packet to _received_packets.

When [interpret_packets\(\)](#) is called is read all the currently active [RxStream](#) instances, and uses this function to enqueue a packet if the stream matches the type and the topic.

Parameters

<i>packet</i>	The packet to add
---------------	-------------------

The documentation for this class was generated from the following files:

- [src/desert_classes/CBorStream.h](#)
- [src/desert_classes/LibMcuCBorStream.cpp](#)
- [src/desert_classes/NanoCBorStream.cpp](#)

6.15 security::SecurityLayer Class Reference

Public Member Functions

- [SecurityLayer](#) ()
Creates a security layer using compile-time configuration values.
- [SecurityLayer](#) (const [AeadParams](#) &aead_params, KdfAlgorithms kdf, const std::vector< uint8_t > &master_key, const std::vector< uint8_t > &master_salt, size_t piv_size, size_t sender_seq_number, const std::vector< uint8_t > &sender_id, const std::vector< uint8_t > &receiver_id)
Creates a security layer from explicitly supplied parameters.
- [SecurityResult](#) [wrap](#) (uint8_t *data, size_t data_len, size_t data_max_len, uint8_t **cose_ptr, size_t *cose_len)
Encrypts and serializes an application payload as a COSE message.
- [SecurityResult](#) [unwrap](#) (uint8_t *data, size_t data_len, size_t data_max_len, size_t *new_data_len)
Decodes, authenticates, and decrypts a COSE message.

6.15.1 Constructor & Destructor Documentation

6.15.1.1 SecurityLayer() [1/2]

```
security::SecurityLayer::SecurityLayer ()
```

Creates a security layer using compile-time configuration values.

Exceptions

<code>std::runtime_error</code>	If identifiers or key material are invalid, or if the security context cannot be derived.
---------------------------------	---

6.15.1.2 SecurityLayer() [2/2]

```
security::SecurityLayer::SecurityLayer (
    const AeadParams & aead_params,
    KdfAlgorithms kdf,
    const std::vector< uint8_t > & master_key,
    const std::vector< uint8_t > & master_salt,
    size_t piv_size,
    size_t sender_seq_number,
    const std::vector< uint8_t > & sender_id,
    const std::vector< uint8_t > & receiver_id)
```

Creates a security layer from explicitly supplied parameters.

Parameters

<code>aead_params</code>	Parameters of the AEAD algorithm to use.
<code>kdf</code>	Key-derivation algorithm to use.
<code>master_key</code>	Master secret used to derive encryption keys.

<i>master_salt</i>	Salt used during key derivation.
<i>piv_size</i>	Size, in bytes, of the Partial IV carried in messages.
<i>sender_seq_number</i>	Initial sender sequence number.
<i>sender_id</i>	Sender identifier used to derive the sender key.
<i>receiver_id</i>	Receiver identifier used to derive the receiver key.

Exceptions

<i>std::runtime_error</i>	If the security context cannot be derived.
---------------------------	--

6.15.2 Member Function Documentation

6.15.2.1 unwrap()

```
SecurityResult security::SecurityLayer::unwrap (
    uint8_t * data,
    size_t data_len,
    size_t data_max_len,
    size_t * new_data_len)
```

Decodes, authenticates, and decrypts a COSE message.

If stateless compression is enabled, the method first reconstructs the standard COSE serialization. It then decrypts the message using the receiver key and copies the recovered plaintext to `data`.

Parameters

<i>data</i>	Buffer containing the received COSE message. On success, it is overwritten with the plaintext payload.
<i>data_len</i>	Length of the received message in bytes.
<i>data_max_len</i>	Total capacity of <code>data</code> in bytes.
<i>new_data_len</i>	Output length of the recovered plaintext in bytes.

Returns

OK on success; UNWRAP_ERROR if parsing, authentication, or decryption fails; otherwise a specific processing error.

6.15.2.2 wrap()

```
SecurityResult security::SecurityLayer::wrap (
    uint8_t * data,
    size_t data_len,
    size_t data_max_len,
    uint8_t ** cose_ptr,
    size_t * cose_len)
```

Encrypts and serializes an application payload as a COSE message.

Generates a Partial IV from the current sender sequence number, adds it as an unprotected COSE header, encrypts the payload with the sender key, and optionally applies the stateless COSE compression format.

On success, the sender sequence number is incremented.

Parameters

<i>data</i>	Buffer containing the plaintext payload and receiving output.
<i>data_len</i>	Length of the plaintext payload in bytes.
<i>data_max_len</i>	Total capacity of <i>data</i> in bytes.
<i>cose_ptr</i>	Output pointer to the beginning of the encoded message.
<i>cose_len</i>	Output length of the encoded COSE message in bytes.

Returns

OK on success; otherwise an error describing the failed step.

Note

The caller must ensure that *data* has sufficient capacity for the resulting serialized message.

The documentation for this class was generated from the following files:

- [src/desert_classes/security/SecurityLayer.h](#)
- [src/desert_classes/security/SecurityLayer.cpp](#)

6.16 TcpDaemon Class Reference

Public Member Functions

- [bool init](#) (int port)
Initialize the socket communication.

Static Public Member Functions

- [static std::vector< uint8_t > read_packet](#) ()
Read a packet from the `_rx_packets` member as vector of bytes.
- [static void enqueue_packet](#) (std::vector< uint8_t > packet)
Enqueue a packet in the `_tx_packets` member as vector of bytes.

6.16.1 Member Function Documentation

6.16.1.1 enqueue_packet()

```
void TcpDaemon::enqueue_packet (
    std::vector< uint8_t > packet) [static]
```

Enqueue a packet in the `_tx_packets` member as vector of bytes.

This function is used by the various TxStream instances contained in publishers, clients and services.

Parameters

<i>packet</i>	The packet that has to be sent through the DESERT stack
---------------	---

6.16.1.2 init()

```
bool TcpDaemon::init (
    int port)
```

Initialize the socket communication.

This function allows the middleware to establish a connection to the DESERT stack through a TCP socket.

Parameters

<i>port</i>	The TCP port of the DESERT application layer
-------------	--

6.16.1.3 read_packet()

```
std::vector< uint8_t > TcpDaemon::read_packet () [static]
```

Read a packet from the `_rx_packets` member as vector of bytes.

This function is used by the various RxStream instances contained in subscribers, clients and services.

Returns

The packet that was read from the DESERT stack

The documentation for this class was generated from the following files:

- `src/desert_classes/TcpDaemon.h`
- `src/desert_classes/TcpDaemon.cpp`

6.17 TopicsConfig Class Reference

Static Public Member Functions

- static void [load_configuration](#) ()
Initialize the configuration.
- static uint8_t [get_topic_identifier](#) (std::string name)
Get topic's identifier from configuration.
- static std::string [get_identifier_topic](#) (uint8_t identifier)
Get identifier's topic from configuration.

6.17.1 Member Function Documentation

6.17.1.1 [get_identifier_topic\(\)](#)

```
std::string TopicsConfig::get_identifier_topic (  
    uint8_t identifier) [static]
```

Get identifier's topic from configuration.

This function returns the topic associated to an identifier if it exists, otherwise returns an empty string.

Returns

The identifier topic

6.17.1.2 [get_topic_identifier\(\)](#)

```
uint8_t TopicsConfig::get_topic_identifier (  
    std::string name) [static]
```

Get topic's identifier from configuration.

This function returns the identifier associated to a topic if it exists, otherwise returns zero.

Returns

The topic identifier

6.17.1.3 [load_configuration\(\)](#)

```
void TopicsConfig::load_configuration () [static]
```

Initialize the configuration.

This function reads the configuration file from: env variable: ROS_ALLOWED_TOPICS_CONFIG or from: `./ros_↔
allowed_topics.conf`. If not present, a warning will be displayed.

The documentation for this class was generated from the following files:

- `src/desert_classes/TopicsConfig.h`
- `src/desert_classes/TopicsConfig.cpp`

6.18 cbor::TxStream Class Reference

Classes

- struct [WRITER](#)
- struct [ERROR](#)

Public Member Functions

- [TxStream](#) (uint8_t stream_type, std::string stream_name, uint8_t stream_identifier)
Create a transmission stream.
- void [start_transmission](#) (uint64_t sequence_id)
Tell the stream to create a new packet.
- void [start_transmission](#) ()
Tell the stream to create a new packet.
- void [end_transmission](#) ()
Tell the stream to send down the packet.
- [TxStream](#) & [operator<<](#) (const uint64_t n)
Encode uint64.
- [TxStream](#) & [operator<<](#) (const uint32_t n)
Encode uint32.
- [TxStream](#) & [operator<<](#) (const uint16_t n)
Encode uint16.
- [TxStream](#) & [operator<<](#) (const uint8_t n)
Encode uint8.
- [TxStream](#) & [operator<<](#) (const int64_t n)
Encode int64.
- [TxStream](#) & [operator<<](#) (const int32_t n)
Encode int32.
- [TxStream](#) & [operator<<](#) (const int16_t n)
Encode int16.
- [TxStream](#) & [operator<<](#) (const int8_t n)
Encode int8.
- [TxStream](#) & [operator<<](#) (const char n)
Encode char.
- [TxStream](#) & [operator<<](#) (const float f)
Encode float.
- [TxStream](#) & [operator<<](#) (const double d)
Encode double.
- [TxStream](#) & [operator<<](#) (const std::string s)
Encode string.
- [TxStream](#) & [operator<<](#) (const std::u16string s)
Encode u16string.
- [TxStream](#) & [operator<<](#) (const bool b)
Encode bool.
- template<typename T>
[TxStream](#) & [operator<<](#) (const std::vector< T > v)
Encode vector.
- [TxStream](#) & [operator<<](#) (const std::vector< bool > v)
Encode bool vector.
- template<typename T>
[TxStream](#) & [serialize_sequence](#) (const T *items, size_t size)
Serialize a sequence of uniform elements.

6.18.1 Constructor & Destructor Documentation

6.18.1.1 TxStream()

```
cbor::TxStream::TxStream (
    uint8_t stream_type,
    std::string stream_name,
    uint8_t stream_identifier)
```

Create a transmission stream.

Parameters

<i>stream_type</i>	Type of the object using the current instance
<i>stream_name</i>	Name of the topic or the service to which the communication belongs
<i>stream_identifier</i>	Identifier of the topic or the service read from configuration

6.18.2 Member Function Documentation

6.18.2.1 end_transmission()

```
void cbor::TxStream::end_transmission ()
```

Tell the stream to send down the packet.

When the transmission is finished the packet is stored in the static member of [TcpDaemon](#) in order to be sent to DESERT.

6.18.2.2 operator<<() [1/16]

```
TxStream & cbor::TxStream::operator<< (
    const bool b)
```

Encode bool.

Parameters

<i>b</i>	Field to encode
----------	-----------------

6.18.2.3 operator<<() [2/16]

```
TxStream & cbor::TxStream::operator<< (  
    const char n)
```

Encode char.

Parameters

<i>n</i>	Field to encode
----------	-----------------

6.18.2.4 operator<<() [3/16]

```
TxStream & cbor::TxStream::operator<< (  
    const double d)
```

Encode double.

Parameters

<i>d</i>	Field to encode
----------	-----------------

6.18.2.5 operator<<() [4/16]

```
TxStream & cbor::TxStream::operator<< (  
    const float f)
```

Encode float.

Parameters

<i>f</i>	Field to encode
----------	-----------------

6.18.2.6 operator<<() [5/16]

```
TxStream & cbor::TxStream::operator<< (  
    const int16_t n)
```

Encode int16.

Parameters

<i>n</i>	Field to encode
----------	-----------------

6.18.2.7 operator<<() [6/16]

```
TxStream & cbor::TxStream::operator<< (  
    const int32_t n)
```

Encode int32.

Parameters

<i>n</i>	Field to encode
----------	-----------------

6.18.2.8 operator<<() [7/16]

```
TxStream & cbor::TxStream::operator<< (  
    const int64_t n)
```

Encode int64.

Parameters

<i>n</i>	Field to encode
----------	-----------------

6.18.2.9 operator<<() [8/16]

```
TxStream & cbor::TxStream::operator<< (  
    const int8_t n)
```

Encode int8.

Parameters

<i>n</i>	Field to encode
----------	-----------------

6.18.2.10 operator<<() [9/16]

```
TxStream & cbor::TxStream::operator<< (  
    const std::string s)
```

Encode string.

Parameters

<i>s</i>	Field to encode
----------	-----------------

6.18.2.11 operator<<() [10/16]

```
TxStream & cbor::TxStream::operator<< (  
    const std::u16string s)
```

Encode u16string.

Parameters

<i>s</i>	Field to encode
----------	-----------------

6.18.2.12 operator<<() [11/16]

```
TxStream & cbor::TxStream::operator<< (  
    const std::vector< bool > v)
```

Encode bool vector.

Parameters

<i>v</i>	Field to encode
----------	-----------------

6.18.2.13 operator<<() [12/16]

```
template<typename T>  
TxStream & cbor::TxStream::operator<< (  
    const std::vector< T > v) [inline]
```

Encode vector.

Parameters

<i>v</i>	Field to encode
----------	-----------------

6.18.2.14 operator<<() [13/16]

```
TxStream & cbor::TxStream::operator<< (  
    const uint16_t n)
```

Encode uint16.

Parameters

<i>n</i>	Field to encode
----------	-----------------

6.18.2.15 operator<<() [14/16]

```
TxStream & cbor::TxStream::operator<< (
    const uint32_t n)
```

Encode uint32.

Parameters

<i>n</i>	Field to encode
----------	-----------------

6.18.2.16 operator<<() [15/16]

```
TxStream & cbor::TxStream::operator<< (
    const uint64_t n)
```

Encode uint64.

Parameters

<i>n</i>	Field to encode
----------	-----------------

6.18.2.17 operator<<() [16/16]

```
TxStream & cbor::TxStream::operator<< (
    const uint8_t n)
```

Encode uint8.

Parameters

<i>n</i>	Field to encode
----------	-----------------

6.18.2.18 serialize_sequence()

```
template<typename T>
TxStream & cbor::TxStream::serialize_sequence (
    const T * items,
    size_t size) [inline]
```

Serialize a sequence of uniform elements.

Parameters

<i>items</i>	Pointer to the first element
<i>size</i>	Size of the items array

6.18.2.19 start_transmission() [1/2]

```
void cbor::TxStream::start_transmission ()
```

Tell the stream to create a new packet.

Every time a transmission is started, a new empty packet must be generated and saved as a private member. Then type and topic name are put in front of the data.

6.18.2.20 start_transmission() [2/2]

```
void cbor::TxStream::start_transmission (
    uint64_t sequence_id)
```

Tell the stream to create a new packet.

Every time a transmission is started, a new empty packet must be generated and saved as a private member. Then type, service name and sequence id are put in front of the data.

Parameters

<i>sequence_id</i>	The id of the client service communication
--------------------	--

The documentation for this class was generated from the following files:

- src/desert_classes/[CBorStream.h](#)
- src/desert_classes/LibMcuCBorStream.cpp
- src/desert_classes/NanoCBorStream.cpp

6.19 cbor::TxStream::WRITER Struct Reference**Public Attributes**

- cbor_writer_t **libmcu_writer**
- nanocbor_encoder_t **nanocbor_encoder**

The documentation for this struct was generated from the following files:

- src/desert_classes/LibMcuCBorStream.cpp
- src/desert_classes/NanoCBorStream.cpp

Chapter 7

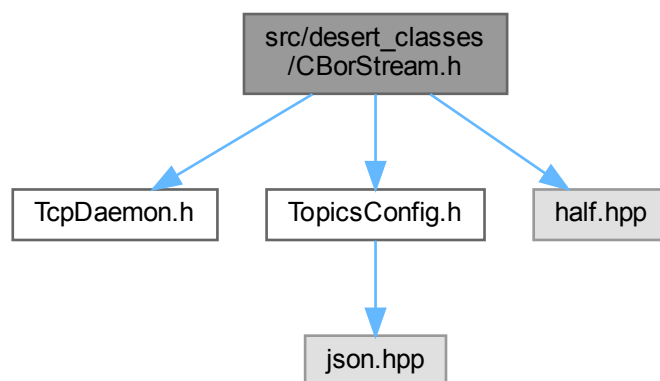
File Documentation

7.1 src/desert_classes/CBORStream.h File Reference

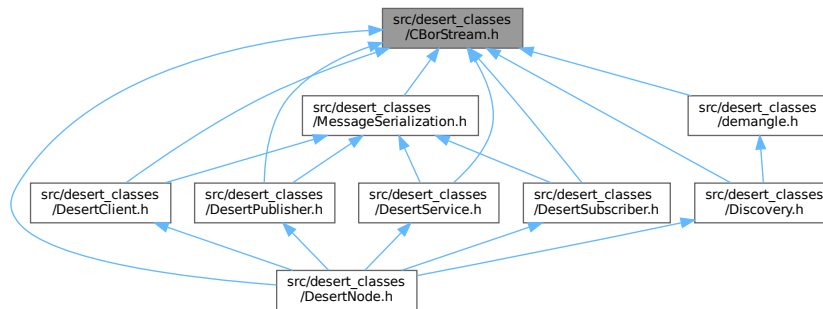
Classes used to convert data types into a CBOR encoded stream.

```
#include "TcpDaemon.h"  
#include "TopicsConfig.h"  
#include "half.hpp"
```

Include dependency graph for CBORStream.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [CircularQueue< T, MaxLen, Container >](#)
- class [cbor::TxStream](#)
- class [cbor::RxStream](#)

Macros

- `#define PUBLISHER_TYPE 0`
- `#define SUBSCRIBER_TYPE 1`
- `#define CLIENT_TYPE 2`
- `#define SERVICE_TYPE 3`
- `#define MAX_BUFFER_CAPACITY 100`

7.1.1 Detailed Description

Classes used to convert data types into a CBOR encoded stream.

In order to perform a socket communication different data types needs to be encoded into binary representations so they can be sent through the same channel. CBOR fits perfectly with the DESERT requirements because only a minimal overhead is introduced in the stream and all the data types are sent using only the minimal quantity of bytes possible.

Author

Prof. Davide Costa

7.2 CborStream.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00033
00034 #ifndef CBORSTREAM_H_
00035 #define CBORSTREAM_H_
00036
00037 #include "TcpDaemon.h"
00038 #include "TopicsConfig.h"
00039
00041
00042 #include <map>
00043 #include <queue>
00044 #include <utility>
00045 #include <vector>
00046 #include <string>
00047 #include <locale>
00048 #include <codecvt>
00049 #include <cstdint>
00050 #include <cstdio>
00051 #include <mutex>
00052
00054
00055 #include "half.hpp"
00056
00057 #define PUBLISHER_TYPE 0
00058 #define SUBSCRIBER_TYPE 1
00059 #define CLIENT_TYPE 2
00060 #define SERVICE_TYPE 3
00061
00062 #define MAX_BUFFER_CAPACITY 100
00063
00064 template <typename T, int MaxLen, typename Container=std::deque<T>
00065 class CircularQueue : public std::queue<T, Container> {
00066 public:
00067     void push(const T& value)
00068     {
00069         if (this->size() == MaxLen)
00070         {
00071             this->c.pop_front();
00072         }
00073         std::queue<T, Container>::push(value);
00074     }
00075 };
00076
00077 namespace cbor
00078 {
00079
00080 class TxStream
00081 {
00082 public:
00083     TxStream(uint8_t stream_type, std::string stream_name, uint8_t stream_identifier);
00084
00086     void start_transmission(uint64_t sequence_id);
00087     void start_transmission();
00088     void end_transmission();
00089
00091     TxStream & operator<<(const uint64_t n);
00092     TxStream & operator<<(const uint32_t n);
00093     TxStream & operator<<(const uint16_t n);
00094     TxStream & operator<<(const uint8_t n);
00095     TxStream & operator<<(const int64_t n);
00096     TxStream & operator<<(const int32_t n);
00097     TxStream & operator<<(const int16_t n);
00098     TxStream & operator<<(const int8_t n);
00099     TxStream & operator<<(const char n);

```

```

00167     TxStream & operator«(const float f);
00172     TxStream & operator«(const double d);
00177     TxStream & operator«(const std::string s);
00182     TxStream & operator«(const std::ul6string s);
00187     TxStream & operator«(const bool b);
00188
00193     template<typename T>
00194     inline TxStream & operator«(const std::vector<T> v)
00195     {
00196         *this « static_cast<const uint32_t>(v.size());
00197         return serialize_sequence(v.data(), v.size());
00198     }
00199
00204     TxStream & operator«(const std::vector<bool> v);
00205
00211     template<typename T>
00212     inline TxStream & serialize_sequence(const T * items, size_t size)
00213     {
00214         for (size_t i = 0; i < size; ++i)
00215         {
00216             *this « items[i];
00217         }
00218         return *this;
00219     }
00220
00221 private:
00222     // Forward declarations to abstract the implementation
00223     struct WRITER;
00224     struct ERROR;
00225
00226     uint8_t _stream_type;
00227     std::string _stream_name;
00228     uint8_t _stream_identifier;
00229
00230     bool _overflow;
00231     uint8_t * _packet;
00232     WRITER * _writer;
00233
00234     void new_packet();
00235     void handle_overrun(ERROR result);
00236
00237     std::string toUTF8(const std::ul6string source);
00238
00239 };
00240
00241 class RxStream
00242 {
00243 public:
00251     RxStream(uint8_t stream_type, std::string stream_name, uint8_t stream_identifier);
00252
00256     ~RxStream();
00257
00267     bool data_available(int64_t sequence_id = 0);
00268
00275     void clear_buffer();
00276
00281     RxStream & operator»(uint64_t & n);
00286     RxStream & operator»(uint32_t & n);
00291     RxStream & operator»(uint16_t & n);
00296     RxStream & operator»(uint8_t & n);
00301     RxStream & operator»(int64_t & n);
00306     RxStream & operator»(int32_t & n);
00311     RxStream & operator»(int16_t & n);
00316     RxStream & operator»(int8_t & n);
00317
00322     RxStream & operator»(char & n);
00327     RxStream & operator»(float & f);
00332     RxStream & operator»(double & d);
00337     RxStream & operator»(std::string & s);
00342     RxStream & operator»(std::ul6string & s);
00347     RxStream & operator»(bool & b);
00348
00353     template<typename T>
00354     inline RxStream & operator»(std::vector<T> & v)
00355     {
00356         uint32_t size;
00357         *this » size;
00358         v.resize(size);
00359
00360         return deserialize_sequence(v.data(), size);
00361     }
00362
00367     RxStream & operator»(std::vector<bool> & v);
00368
00374     template<typename T>
00375     inline RxStream & deserialize_sequence(T * items, size_t size)
00376     {

```

```

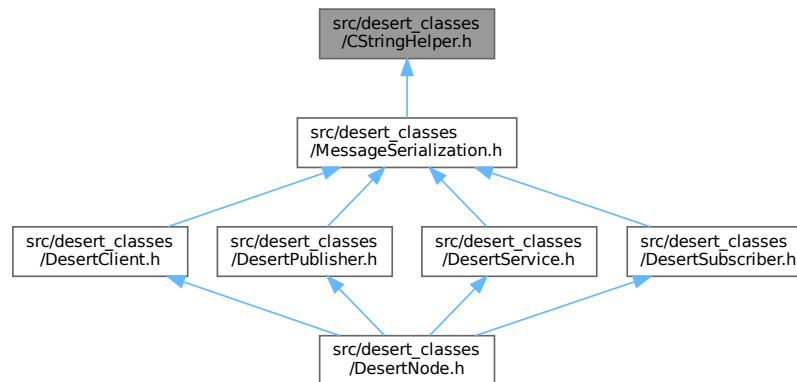
00377         for (size_t i = 0; i < size; ++i)
00378         {
00379             *this » items[i];
00380         }
00381         return *this;
00382     }
00383
00384
00389     uint8_t get_type() const;
00394     std::string get_name() const;
00399     uint8_t get_identifier() const;
00400
00410     void push_packet(std::vector<std::pair<void *, int>» packet);
00411
00419     static void interpret_packets();
00420
00421 private:
00422     // Forward declarations to abstract the implementation
00423     struct ITEM;
00424
00425     uint8_t _stream_type;
00426     std::string _stream_name;
00427     uint8_t _stream_identifier;
00428
00429     size_t _buffered_iterator;
00430
00431     // packets: <packet <field, field_type>
00432     std::vector<std::pair<void *, int>» _buffered_packet;
00433     CircularQueue<std::vector<std::pair<void *, int>», MAX_BUFFER_CAPACITY> _received_packets;
00434
00435     static const std::map<int, int> _stream_type_match_map;
00436     static std::vector<RxStream *> _listening_streams;
00437
00438     union _cbor_value {
00439         int8_t i8;
00440         int16_t i16;
00441         int32_t i32;
00442         int64_t i64;
00443         float f32;
00444         double f64;
00445         uint8_t *bin;
00446         char *str;
00447         uint8_t str_copy[128];
00448     };
00449
00450     static std::mutex _rx_mutex;
00451
00452     static std::pair<void *, int> interpret_field(ITEM * items, size_t i, union _cbor_value & val);
00453     static void destroy_interpreted_field(std::pair<void *, int>& field);
00454     std::u16string toUTF16(const std::string source);
00455 };
00456
00457 } // namespace cbor
00458
00459
00460 #endif

```

7.3 src/desert_classes/CStringHelper.h File Reference

Namespace containing C sequence handling functions.

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [CStringHelper](#)
Namespace containing C sequence handling functions.

Functions

- [std::string CStringHelper::convert_to_std_string](#) (void *str)
Convert a `rosidl_runtime_c_String` into `std::string`.
- [std::vector< std::string > CStringHelper::convert_to_std_vector_string](#) (void *str_array, size_t size)
Convert a `rosidl_runtime_c_String` into a vector of `std::string`.
- [std::vector< std::string > CStringHelper::convert_sequence_to_std_vector_string](#) (void *str_seq)
Convert a `rosidl_runtime_c_String__Sequence` into a vector of `std::string`.
- [std::u16string CStringHelper::convert_to_std_u16string](#) (void *str)
Convert a `rosidl_runtime_c_U16String` into `std::u16string`.
- [std::vector< std::u16string > CStringHelper::convert_to_std_vector_u16string](#) (void *str_array, size_t size)
Convert a `rosidl_runtime_c_U16String` into a vector of `std::u16string`.
- [std::vector< std::u16string > CStringHelper::convert_sequence_to_std_vector_u16string](#) (void *str_seq)
Convert a `rosidl_runtime_c_U16String__Sequence` into a vector of `std::u16string`.
- [void CStringHelper::assign_string](#) (std::string str, void *field)
Assigning to a `rosidl_runtime_c_String` the value contained in a `std::string`.
- [void CStringHelper::assign_vector_string](#) (std::vector< std::string > cpp_string_vector, void *str_array, size_t size)
Assigning to a `rosidl_runtime_c_String` the value contained in a vector of `std::string`.
- [void CStringHelper::assign_vector_string_to_sequence](#) (std::vector< std::string > cpp_string_vector, void *str_seq)
Assigning to a `rosidl_runtime_c_String__Sequence` the value contained in a vector of `std::string`.
- [void CStringHelper::assign_u16string](#) (std::u16string str, void *field)
Assigning to a `rosidl_runtime_c_U16String` the value contained in a `std::u16string`.
- [void CStringHelper::assign_vector_u16string](#) (std::vector< std::u16string > cpp_string_vector, void *str_array, size_t size)
Assigning to a `rosidl_runtime_c_U16String` the value contained in a vector of `std::u16string`.
- [void CStringHelper::assign_vector_u16string_to_sequence](#) (std::vector< std::u16string > cpp_string_vector, void *str_seq)
Assigning to a `rosidl_runtime_c_U16String__Sequence` the value contained in a vector of `std::u16string`.

7.3.1 Detailed Description

Namespace containing C sequence handling functions.

The C data type implementation is more complicated than the C++ one, because complex types like vectors have to be manually managed and this header contains functions to convert C strings and generic sequences into respectively C++ strings and vectors.

Author

Prof. Davide Costa

7.4 CStringHelper.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00031
00032 #ifndef CSTRING_HELPER_H_
00033 #define CSTRING_HELPER_H_
00034
00036
00037 #include "rosidl_runtime_c/ul6string.h"
00038 #include "rosidl_runtime_c/string.h"
00039 #include "rosidl_runtime_c/ul6string_functions.h"
00040 #include "rosidl_runtime_c/string_functions.h"
00041
00042 #include <stdexcept>
00043 #include <vector>
00044 #include <string>
00045
00047
00056 namespace CStringHelper
00057 {
00066     std::string convert_to_std_string(void * str);
00077     std::vector<std::string> convert_to_std_vector_string(void * str_array, size_t size);
00087     std::vector<std::string> convert_sequence_to_std_vector_string(void * str_seq);
00088
00097     std::ul6string convert_to_std_ul6string(void * str);
00108     std::vector<std::ul6string> convert_to_std_vector_ul6string(void * str_array, size_t size);
00118     std::vector<std::ul6string> convert_sequence_to_std_vector_ul6string(void * str_seq);
00119
00129     void assign_string(std::string str, void * field);
00140     void assign_vector_string(std::vector<std::string> cpp_string_vector, void * str_array, size_t
size);
00150     void assign_vector_string_to_sequence(std::vector<std::string> cpp_string_vector, void * str_seq);
00151
00161     void assign_ul6string(std::ul6string str, void * field);
00172     void assign_vector_ul6string(std::vector<std::ul6string> cpp_string_vector, void * str_array, size_t
size);
00182     void assign_vector_ul6string_to_sequence(std::vector<std::ul6string> cpp_string_vector, void *
str_seq);
00183 }
00184
00185
00186 #endif

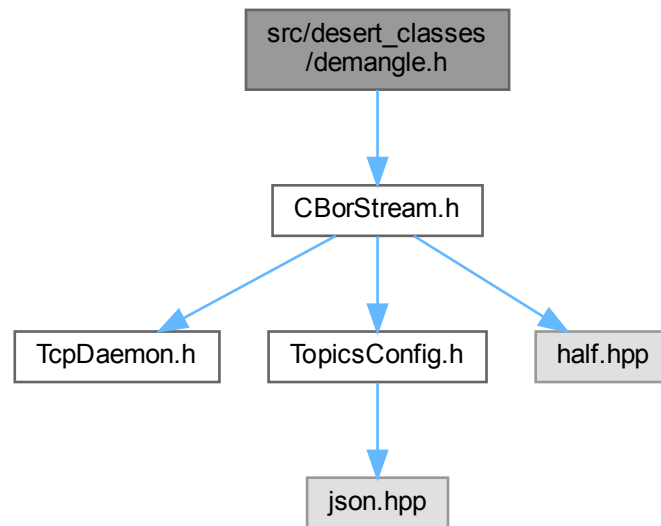
```

7.5 src/desert_classes/demangle.h File Reference

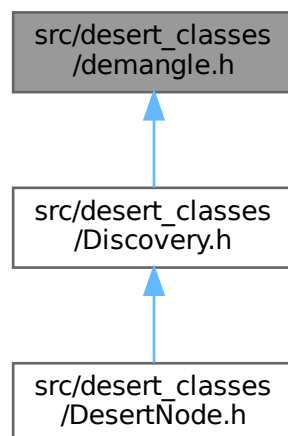
Functions used to demangle topic names during discovery operations.

```
#include "CBorStream.h"
```

Include dependency graph for demangle.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [Discovery](#)
Namespace containing discovery functions.

Typedefs

- using **Discovery::DemangleFunction** = std::string (*)(const std::string &)

Functions

- char * **Discovery::integer_to_string** (int x)
- std::string [Discovery::resolve_prefix](#) (const std::string &name, const std::string &prefix)
Resolve a prefix.
- std::string [Discovery::demangle_publisher_from_topic](#) (const std::string &topic_name)
Demangle a publisher.
- std::string [Discovery::demangle_subscriber_from_topic](#) (const std::string &topic_name)
Demangle a subscriber.
- std::string [Discovery::demangle_topic](#) (const std::string &topic_name)
Demangle a topic.
- std::string [Discovery::demangle_service_request_from_topic](#) (const std::string &topic_name)
Demangle a service request.
- std::string [Discovery::demangle_service_reply_from_topic](#) (const std::string &topic_name)
Demangle a service reply.
- std::string [Discovery::demangle_service_from_topic](#) (const std::string &topic_name)
Demangle a service.
- std::string [Discovery::identity_demangle](#) (const std::string &name)
No demangle.

Variables

- const char *const **Discovery::ros_topic_publisher_prefix** = integer_to_string(PUBLISHER_TYPE)
- const char *const **Discovery::ros_topic_subscriber_prefix** = integer_to_string(SUBSCRIBER_TYPE)
- const char *const **Discovery::ros_service_requester_prefix** = integer_to_string(CLIENT_TYPE)
- const char *const **Discovery::ros_service_response_prefix** = integer_to_string(SERVICE_TYPE)

7.5.1 Detailed Description

Functions used to demangle topic names during discovery operations.

Demangle functions allows to extract topic names and type names of each entity stored in the common context implementation. Since in this object the informations are divided in writers and readers, they must be converted in publishers, subscribers, clients and services.

Author

Prof. Davide Costa

7.6 demangle.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00033
00034 #ifndef DEMANGLE_H_
00035 #define DEMANGLE_H_
00036
00038
00039 #include <algorithm>
00040 #include <string>
00041 #include <vector>
00042
00043 #include "rcpputils/find_and_replace.hpp"
00044 #include "rcutils/logging_macros.h"
00045 #include "rcutils/types.h"
00046
00048
00049 #include "CBorStream.h"
00050
00061 namespace Discovery
00062 {
00063
00064     char * integer_to_string(int x);
00065
00066     const char * const ros_topic_publisher_prefix = integer_to_string(PUBLISHER_TYPE);
00067     const char * const ros_topic_subscriber_prefix = integer_to_string(SUBSCRIBER_TYPE);
00068     const char * const ros_service_requester_prefix = integer_to_string(CLIENT_TYPE);
00069     const char * const ros_service_response_prefix = integer_to_string(SERVICE_TYPE);
00070
00080     std::string resolve_prefix(const std::string & name, const std::string & prefix);
00081
00090     std::string demangle_publisher_from_topic(const std::string & topic_name);
00091
00100     std::string demangle_subscriber_from_topic(const std::string & topic_name);
00101
00110     std::string demangle_topic(const std::string & topic_name);
00111
00120     std::string demangle_service_request_from_topic(const std::string & topic_name);
00121
00130     std::string demangle_service_reply_from_topic(const std::string & topic_name);
00131
00140     std::string demangle_service_from_topic(const std::string & topic_name);
00141
00150     std::string identity_demangle(const std::string & name);
00151
00152     using DemangleFunction = std::string (*)(const std::string &);
00153
00154 }
00155
00156 #endif // DEMANGLE_H_

```

7.7 src/desert_classes/DesertClient.h File Reference

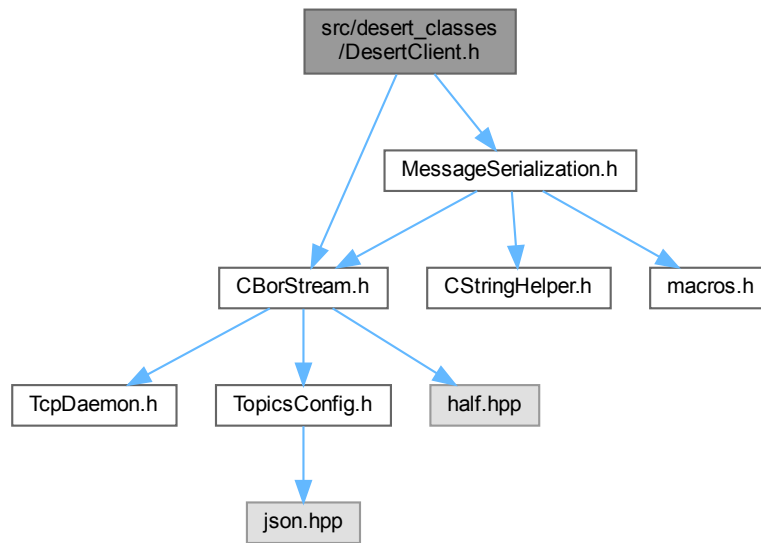
Implementation of the Client structure for DESERT.

```

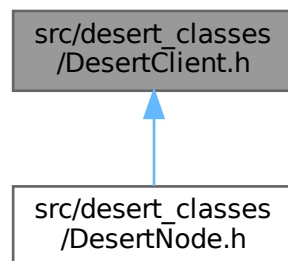
#include "CBorStream.h"
#include "MessageSerialization.h"

```

Include dependency graph for DesertClient.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [DesertClient](#)

7.7.1 Detailed Description

Implementation of the Client structure for DESERT.

The [DesertClient](#) class is used to create instances of the various clients registered by ROS. Each of them contains the informations needed to decode the data structure of the messages in the service and allows to send and receive data through specific public functions.

Author

Prof. Davide Costa

7.8 DesertClient.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00020
00021
00022
00023 #ifndef DESERT_CLIENT_H_
00024 #define DESERT_CLIENT_H_
00025
00026
00027
00028 #include "rosidl_typesupport_introspection_cpp/identifier.hpp"
00029 #include "rosidl_typesupport_introspection_c/identifier.h"
00030 #include "rosidl_typesupport_introspection_cpp/message_introspection.hpp"
00031 #include "rosidl_typesupport_introspection_c/message_introspection.h"
00032 #include "rosidl_typesupport_introspection_cpp/service_introspection.hpp"
00033 #include "rosidl_typesupport_introspection_c/service_introspection.h"
00034 #include "rosidl_typesupport_introspection_cpp/field_types.hpp"
00035 #include "rosidl_typesupport_introspection_c/field_types.h"
00036
00037 #include "rosidl_runtime_c/service_type_support_struct.h"
00038
00039 #include "rmw/types.h"
00040
00041 #include <vector>
00042 #include <string>
00043 #include <regex>
00044
00045
00046 #include "CBorStream.h"
00047 #include "MessageSerialization.h"
00048
00049
00050 class DesertClient
00051 {
00052 public:
00053     DesertClient(std::string service_name, const rosidl_service_type_support_t * type_supports,
00054                 rmw_gid_t gid);
00055
00056     bool has_data();
00057     void send_request(const void * req, int64_t * sequence_id);
00058     void read_response(void * res, rmw_service_info_t * req_header);
00059
00060     rmw_gid_t get_gid();
00061     std::string get_service_name();
00062     std::string get_request_type_name();
00063     std::string get_response_type_name();
00064
00065 private:
00066     uint8_t _id;
00067     rmw_gid_t _gid;
00068     std::string _name;
00069     cbor::TxStream _request_data_stream;
00070     cbor::RxStream _response_data_stream;
00071
00072     int64_t _sequence_id;
00073
00074     int _c_cpp_identifier;
00075     const void * _service;
00076
00077     const void * get_service(const rosidl_service_type_support_t * service_type_support);

```

```

00152     const rosidl_service_type_support_t * get_service_type_support (const rosidl_service_type_support_t
    * type_supports);
00153
00154 };
00155
00156 #endif

```

7.9 src/desert_classes/DesertGuardCondition.h File Reference

Implementation of the GuardCondition structure for DESERT.

Classes

- class [DesertGuardCondition](#)

7.9.1 Detailed Description

Implementation of the GuardCondition structure for DESERT.

The [DesertGuardCondition](#) class is used to handle trigger signals sent from rclcpp in order to break rcl_wait, usually when dealing with multithreading executors.

Author

Prof. Davide Costa

7.10 DesertGuardCondition.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00020
00030
00031 #ifndef DESERT_GUARD_CONDITION_H_
00032 #define DESERT_GUARD_CONDITION_H_
00033
00034
00035
00036 #include <array>
00037 #include <atomic>
00038 #include <cassert>
00039 #include <condition_variable>
00040 #include <mutex>
00041 #include <utility>
00042
00043 #include "rcpputils/thread_safety_annotations.hpp"
00044
00045
00046

```

```

00047 class DesertGuardCondition
00048 {
00049     public:
00053         DesertGuardCondition();
00054
00061         void trigger();
00062
00071         bool has_triggered();
00072
00081         bool get_has_triggered();
00082
00083     private:
00084         std::atomic_bool _has_triggered;
00085 };
00086
00087 #endif

```

7.11 src/desert_classes/DesertNode.h File Reference

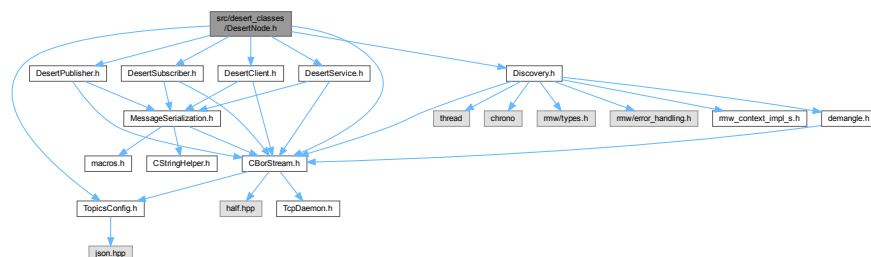
Implementation of the Node structure for DESERT.

```

#include "CBorStream.h"
#include "DesertPublisher.h"
#include "DesertSubscriber.h"
#include "DesertClient.h"
#include "DesertService.h"
#include "Discovery.h"
#include "TopicsConfig.h"

```

Include dependency graph for DesertNode.h:



Classes

- class [DesertNode](#)

7.11.1 Detailed Description

Implementation of the Node structure for DESERT.

The [DesertNode](#) class is used to keep track of all the entities created by a specific node. Each of them is stored in a vector of pointers to the original memory locations mainly used to provide discovery functionalities.

Author

Prof. Davide Costa

7.12 DesertNode.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00031
00032 #ifndef DESERT_NODE_H_
00033 #define DESERT_NODE_H_
00034
00036
00037 #include "rmw/rmw.h"
00038 #include "rmw/types.h"
00039
00040 #include <vector>
00041 #include <string>
00042
00044
00045 #include "CBorStream.h"
00046 #include "DesertPublisher.h"
00047 #include "DesertSubscriber.h"
00048 #include "DesertClient.h"
00049 #include "DesertService.h"
00050 #include "Discovery.h"
00051 #include "TopicsConfig.h"
00052
00053 class DesertNode
00054 {
00055 public:
00063     DesertNode(std::string name, std::string namespace_, rmw_gid_t gid);
00064     ~DesertNode();
00065
00074     void add_publisher(DesertPublisher * pub);
00075
00084     void add_subscriber(DesertSubscriber * sub);
00085
00094     void add_client(DesertClient * cli);
00095
00104     void add_service(DesertService * ser);
00105
00114     void remove_publisher(DesertPublisher * pub);
00115
00124     void remove_subscriber(DesertSubscriber * sub);
00125
00134     void remove_client(DesertClient * cli);
00135
00144     void remove_service(DesertService * ser);
00145
00153     rmw_gid_t get_gid();
00154
00155 private:
00156     rmw_gid_t _gid;
00157     std::string _name;
00158     std::string _namespace;
00159     cbor::TxStream _discovery_beacon_data_stream;
00160     cbor::RxStream _discovery_request_data_stream;
00161
00162     std::vector<DesertPublisher *> _publishers;
00163     std::vector<DesertSubscriber *> _subscribers;
00164     std::vector<DesertClient *> _clients;
00165     std::vector<DesertService *> _services;
00166
00167     void publish_all_beacons();
00168
00169     bool _discovery_done;
00170     std::thread _discovery_request_thread;
00171
00172     void _discovery_request();
00173

```

```

00174 };
00175
00176 #endif

```

7.13 src/desert_classes/DesertPublisher.h File Reference

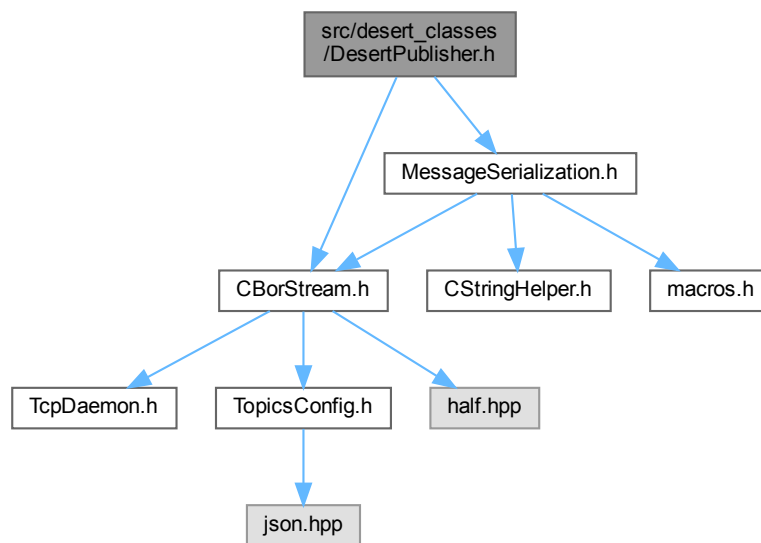
Implementation of the Publisher structure for DESERT.

```

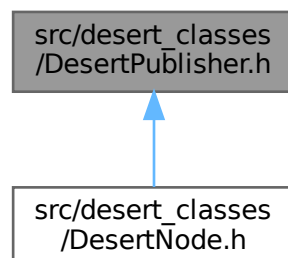
#include "CBorStream.h"
#include "MessageSerialization.h"

```

Include dependency graph for DesertPublisher.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [DesertPublisher](#)

7.13.1 Detailed Description

Implementation of the Publisher structure for DESERT.

The [DesertPublisher](#) class is used to create instances of the various publishers registered by ROS. Each of them contains the informations needed to encode the data structure of the messages in the topic and send them to the stream through specific public functions.

Author

Prof. Davide Costa

7.14 DesertPublisher.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00032
00033 #ifndef DESERT_PUBLISHER_H_
00034 #define DESERT_PUBLISHER_H_
00035
00037
00038 #include "rosidl_typesupport_introspection_cpp/identifier.hpp"
00039 #include "rosidl_typesupport_introspection_c/identifier.h"
00040 #include "rosidl_typesupport_introspection_cpp/message_introspection.hpp"
00041 #include "rosidl_typesupport_introspection_c/message_introspection.h"
00042 #include "rosidl_typesupport_introspection_cpp/service_introspection.hpp"
00043 #include "rosidl_typesupport_introspection_c/service_introspection.h"
00044 #include "rosidl_typesupport_introspection_cpp/field_types.hpp"
00045 #include "rosidl_typesupport_introspection_c/field_types.h"
00046
00047 #include "rosidl_runtime_c/message_type_support_struct.h"
00048
00049 #include "rmw/types.h"
00050
00051 #include <vector>
00052 #include <string>
00053 #include <regex>
00054
00056
00057 #include "CborStream.h"
00058 #include "MessageSerialization.h"
00059
00060 class DesertPublisher
00061 {
00062 public:
00063     DesertPublisher(std::string topic_name, const rosidl_message_type_support_t * type_supports,
00064                   rmw_gid_t gid);
00071
00081     void push(const void * msg);

```

```

00082
00090     rmw_gid_t get_gid();
00098     std::string get_topic_name();
00106     std::string get_type_name();
00107
00108
00109     private:
00110         uint8_t _id;
00111         rmw_gid_t _gid;
00112         std::string _name;
00113         cbor::TxStream _data_stream;
00114
00115         int _c_cpp_identifier;
00116         const void * _members;
00117
00118         const void * get_members(const rosidl_message_type_support_t * type_support);
00119         const rosidl_message_type_support_t * get_type_support(const rosidl_message_type_support_t *
type_supports);
00120
00121 };
00122
00123 #endif

```

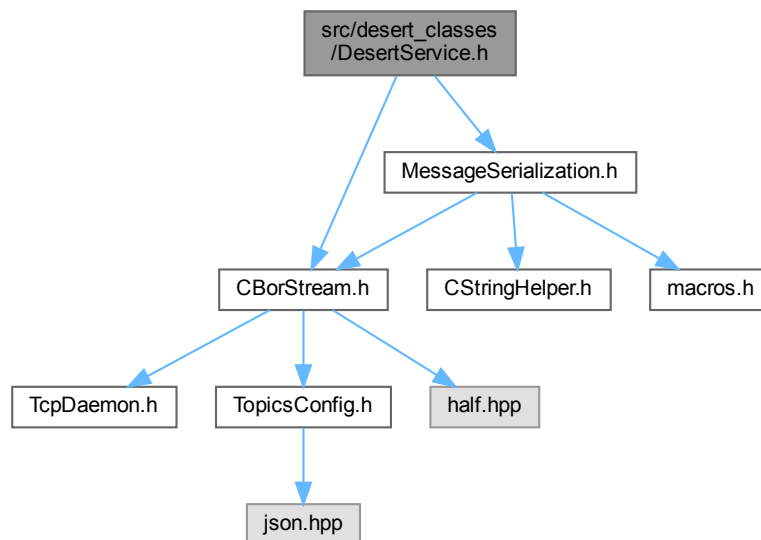
7.15 src/desert_classes/DesertService.h File Reference

Implementation of the Service structure for DESERT.

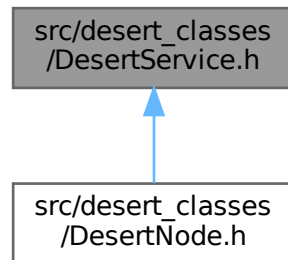
```
#include "CBorStream.h"
```

```
#include "MessageSerialization.h"
```

Include dependency graph for DesertService.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [DesertService](#)

7.15.1 Detailed Description

Implementation of the Service structure for DESERT.

The [DesertService](#) class is used to create instances of the various services registered by ROS. Each of them contains the informations needed to decode the data structure of the messages in the stream and allows to send and receive data through specific public functions.

Author

Prof. Davide Costa

7.16 DesertService.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00032
00033 #ifndef DESERT_SERVICE_H_
00034 #define DESERT_SERVICE_H_
  
```

```

00035
00037
00038 #include "rosidl_typesupport_introspection_cpp/identifier.hpp"
00039 #include "rosidl_typesupport_introspection_c/identifier.h"
00040 #include "rosidl_typesupport_introspection_cpp/message_introspection.hpp"
00041 #include "rosidl_typesupport_introspection_c/message_introspection.h"
00042 #include "rosidl_typesupport_introspection_cpp/service_introspection.hpp"
00043 #include "rosidl_typesupport_introspection_c/service_introspection.h"
00044 #include "rosidl_typesupport_introspection_cpp/field_types.hpp"
00045 #include "rosidl_typesupport_introspection_c/field_types.h"
00046
00047 #include "rosidl_runtime_c/service_type_support_struct.h"
00048
00049 #include "rmw/types.h"
00050
00051 #include <vector>
00052 #include <string>
00053 #include <regex>
00054
00056
00057 #include "CBorStream.h"
00058 #include "MessageSerialization.h"
00059
00060 class DesertService
00061 {
00062     public:
00070     DesertService(std::string service_name, const rosidl_service_type_support_t * type_supports,
00071                 rmw_gid_t gid);
00081     bool has_data();
00092     void read_request(void * req, rmw_service_info_t * req_header);
00103     void send_response(void * res, rmw_request_id_t * req_header);
00104
00112     rmw_gid_t get_gid();
00120     std::string get_service_name();
00128     std::string get_request_type_name();
00136     std::string get_response_type_name();
00137
00138
00139     private:
00140         uint8_t _id;
00141         rmw_gid_t _gid;
00142         std::string _name;
00143         cbor::RxStream _request_data_stream;
00144         cbor::TxStream _response_data_stream;
00145
00146         int64_t _sequence_id;
00147
00148         int _c_cpp_identifier;
00149         const void * _service;
00150
00151         const void * get_service(const rosidl_service_type_support_t * service_type_support);
00152         const rosidl_service_type_support_t * get_service_type_support(const rosidl_service_type_support_t
00153 * type_supports);
00154 };
00155
00156 #endif

```

7.17 src/desert_classes/DesertSubscriber.h File Reference

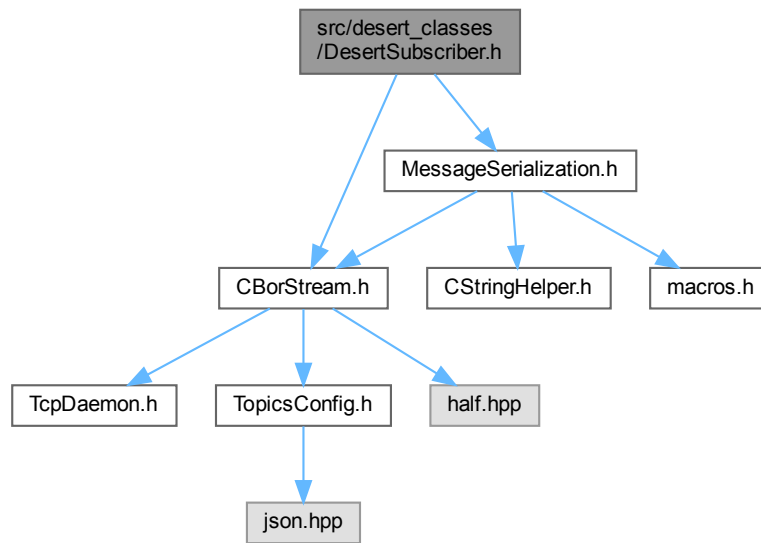
Implementation of the Subscriber structure for DESERT.

```

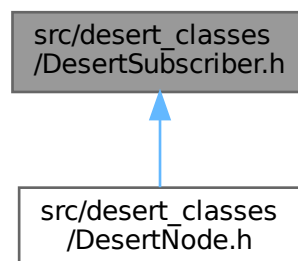
#include "CBorStream.h"
#include "MessageSerialization.h"

```

Include dependency graph for DesertSubscriber.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [DesertSubscriber](#)

7.17.1 Detailed Description

Implementation of the Subscriber structure for DESERT.

The [DesertSubscriber](#) class is used to create instances of the various subscribers registered by ROS. Each of them contains the informations needed to decode the data structure of the messages in the topic through specific public functions.

Author

Prof. Davide Costa

7.18 DesertSubscriber.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00031
00032 #ifndef DESERT_SUBSCRIBER_H_
00033 #define DESERT_SUBSCRIBER_H_
00034
00036
00037 #include "rosidl_typesupport_introspection_cpp/identifier.hpp"
00038 #include "rosidl_typesupport_introspection_c/identifier.h"
00039 #include "rosidl_typesupport_introspection_cpp/message_introspection.hpp"
00040 #include "rosidl_typesupport_introspection_c/message_introspection.h"
00041 #include "rosidl_typesupport_introspection_cpp/service_introspection.hpp"
00042 #include "rosidl_typesupport_introspection_c/service_introspection.h"
00043 #include "rosidl_typesupport_introspection_cpp/field_types.hpp"
00044 #include "rosidl_typesupport_introspection_c/field_types.h"
00045
00046 #include "rosidl_runtime_c/message_type_support_struct.h"
00047
00048 #include "rmw/types.h"
00049
00050 #include <vector>
00051 #include <string>
00052 #include <regex>
00053
00055
00056 #include "CBorStream.h"
00057 #include "MessageSerialization.h"
00058
00059 class DesertSubscriber
00060 {
00061 public:
00062     DesertSubscriber(std::string topic_name, const rosidl_message_type_support_t * type_supports,
00063                     rmw_gid_t gid);
00070
00080     bool has_data();
00090     void read_data(void * msg);
00091
00099     rmw_gid_t get_gid();
00107     std::string get_topic_name();
00115     std::string get_type_name();
00116
00117 private:
00118     uint8_t _id;
00119     rmw_gid_t _gid;
00120     std::string _name;
00121     cbor::RxStream _data_stream;
00122
00123     int _c_cpp_identifier;
00124     const void * _members;
00125
00126     const void * get_members(const rosidl_message_type_support_t * type_support);
00127     const rosidl_message_type_support_t * get_type_support(const rosidl_message_type_support_t *
00128 type_supports);
00129 };
00130
00131 #endif

```

7.19 src/desert_classes/DesertWaitSet.h File Reference

Implementation of the WaitSet structure for DESERT.

Classes

- class [DesertWaitset](#)

7.19.1 Detailed Description

Implementation of the WaitSet structure for DESERT.

Unimplemented class included for future expansions

Author

Prof. Davide Costa

7.20 DesertWaitSet.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00020
00021
00022
00023 #ifndef DESERT_WAIT_SET_H_
00024 #define DESERT_WAIT_SET_H_
00025
00026 class DesertWaitset
00027 {
00028 public:
00029     DesertWaitset()
00030     {}
00031
00032     std::mutex lock;
00033     bool inuse;
00034 };
00035
00036 #endif

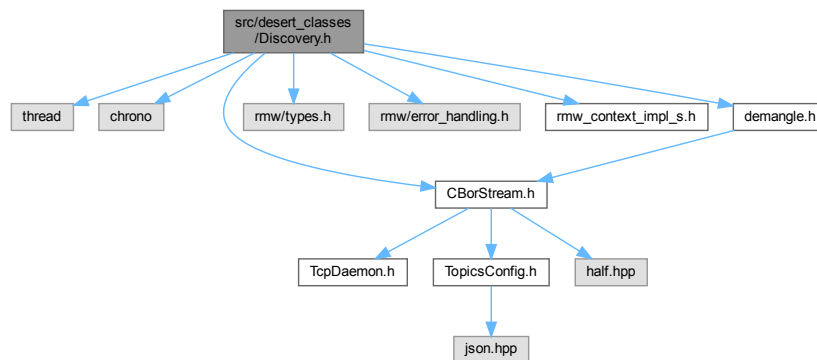
```

7.21 src/desert_classes/Discovery.h File Reference

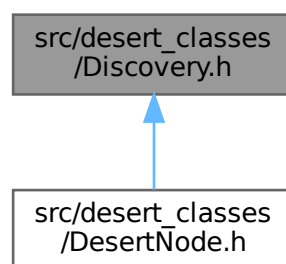
Namespace used to provide discovery functionalities.

```
#include <thread>
#include <chrono>
#include "CBorStream.h"
#include "rmw/types.h"
#include "rmw/error_handling.h"
#include "rmw_context_impl_s.h"
#include "demangle.h"
```

Include dependency graph for Discovery.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [Discovery](#)
Namespace containing discovery functions.

Functions

- void `Discovery::discovery_thread` (`rmw_context_impl_t *impl`)
Thread handling discovery beacons.
- `rmw_ret_t` `Discovery::discovery_thread_start` (`rmw_context_impl_t *impl`)
Initialize the discovery thread.
- `rmw_ret_t` `Discovery::discovery_thread_stop` (`rmw_context_impl_t *impl`)
Stop the discovery thread.
- void `Discovery::send_discovery_beacon` (`cbor::TxStream` stream, `std::string` node_name, `std::string` node_namespace, `int` entity_type, `rmw_gid_t` entity_gid, `std::string` topic_name, `std::string` type_name, `bool` disconnect)
Send a discovery beacon.
- void `Discovery::send_discovery_request` (`cbor::TxStream` stream)
Send a discovery request.

7.21.1 Detailed Description

Namespace used to provide discovery functionalities.

The middleware layer of a ROS stack must implement functionalities used to inform each node of the network structure of the other nodes connected, with their names and topics. Since this operation is quite resource-consuming and the underwater channel has a limited bandwidth, it is possible to disable it.

Author

Prof. Davide Costa

7.22 Discovery.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00033
00034 #include <thread>
00035 #include <chrono>
00036
00037 #include "CBorStream.h"
00038
00039 #include "rmw/types.h"
00040 #include "rmw/error_handling.h"
00041
00042 #include "rmw_context_impl_s.h"
00043
00044 #include "demangle.h"
00045
00046 #ifndef DISCOVERY_H_

```

```

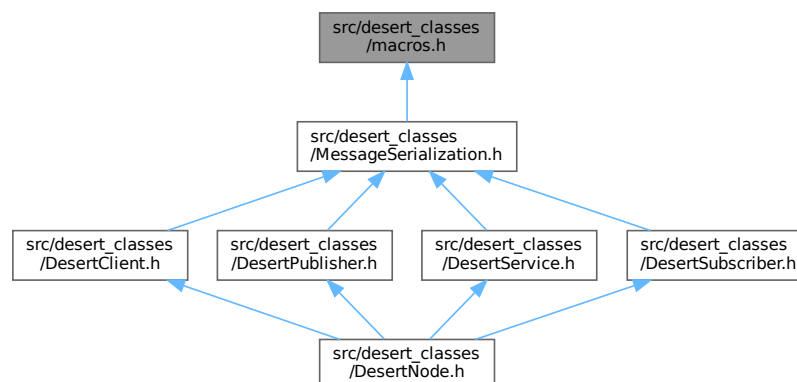
00047 #define DISCOVERY_H_
00048
00059 namespace Discovery
00060 {
00061
00070     void discovery_thread(rmw_context_impl_t * impl);
00071
00081     rmw_ret_t discovery_thread_start(rmw_context_impl_t * impl);
00082
00092     rmw_ret_t discovery_thread_stop(rmw_context_impl_t * impl);
00093
00109     void send_discovery_beacon(cbor::TxStream stream, std::string node_name, std::string node_namespace,
                                int entity_type, rmw_gid_t entity_gid, std::string topic_name, std::string type_name, bool
                                disconnect);
00110
00119     void send_discovery_request(cbor::TxStream stream);
00120
00121 }
00122
00123 #endif

```

7.23 src/desert_classes/macros.h File Reference

Header containing C sequence macros.

This graph shows which files directly or indirectly include this file:



Macros

- #define `SPECIALIZE_GENERIC_C_SEQUENCE(C_NAME, C_TYPE)`

7.23.1 Detailed Description

Header containing C sequence macros.

The C data type implementation is more complicated than the C++ one, because complex types like vectors have to be manually managed and this header contains definitions used to create dynamic element sequences.

Author

Prof. Davide Costa

7.23.2 Macro Definition Documentation

7.23.2.1 SPECIALIZE_GENERIC_C_SEQUENCE

```
#define SPECIALIZE_GENERIC_C_SEQUENCE(
```

```
    C_NAME,
```

```
    C_TYPE)
```

Value:

```
template<> \
struct GenericCSequence<C_TYPE> \
{ \
    using type = rosidl_runtime_c__ ## C_NAME ## __Sequence; \
\
    static void fini(type * sequence) { \
        rosidl_runtime_c__ ## C_NAME ## __Sequence__fini(sequence); \
    } \
\
    static bool init(type * sequence, size_t size) { \
        return rosidl_runtime_c__ ## C_NAME ## __Sequence__init(sequence, size); \
    } \
};
```

7.24 macros.h

[Go to the documentation of this file.](#)

```
00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00031
00032 #ifndef MACROS_H_
00033 #define MACROS_H_
00034
00036
00037 #include "rosidl_runtime_c/primitives_sequence.h"
00038 #include "rosidl_runtime_c/primitives_sequence_functions.h"
00039
00041
00042 #define SPECIALIZE_GENERIC_C_SEQUENCE(C_NAME, C_TYPE) \
00043     template<> \
00044     struct GenericCSequence<C_TYPE> \
00045     { \
00046         using type = rosidl_runtime_c__ ## C_NAME ## __Sequence; \
00047     \
00048         static void fini(type * sequence) { \
00049             rosidl_runtime_c__ ## C_NAME ## __Sequence__fini(sequence); \
00050         } \
00051     \
00052         static bool init(type * sequence, size_t size) { \
00053             return rosidl_runtime_c__ ## C_NAME ## __Sequence__init(sequence, size); \
00054         } \
00055     };
00056
00057 template<typename T>
00058 struct GenericCSequence;
00059
00060 // multiple definitions of ambiguous primitive types
00061 SPECIALIZE_GENERIC_C_SEQUENCE(bool, bool)
00062 SPECIALIZE_GENERIC_C_SEQUENCE(byte, uint8_t)
```

```

00063 SPECIALIZE_GENERIC_C_SEQUENCE(char, char)
00064 SPECIALIZE_GENERIC_C_SEQUENCE(float32, float)
00065 SPECIALIZE_GENERIC_C_SEQUENCE(float64, double)
00066 SPECIALIZE_GENERIC_C_SEQUENCE(int8, int8_t)
00067 SPECIALIZE_GENERIC_C_SEQUENCE(int16, int16_t)
00068 SPECIALIZE_GENERIC_C_SEQUENCE(uint16, uint16_t)
00069 SPECIALIZE_GENERIC_C_SEQUENCE(int32, int32_t)
00070 SPECIALIZE_GENERIC_C_SEQUENCE(uint32, uint32_t)
00071 SPECIALIZE_GENERIC_C_SEQUENCE(int64, int64_t)
00072 SPECIALIZE_GENERIC_C_SEQUENCE(uint64, uint64_t)
00073
00074 #endif // MACROS_HPP_

```

7.25 src/desert_classes/MessageSerialization.h File Reference

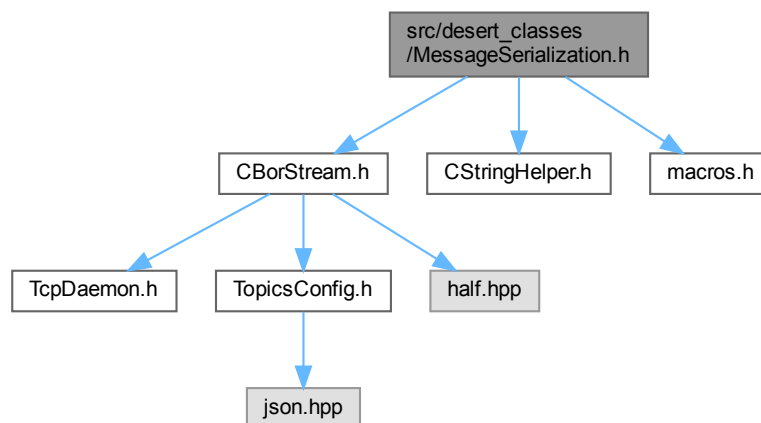
Namespace containing serialization functions.

```

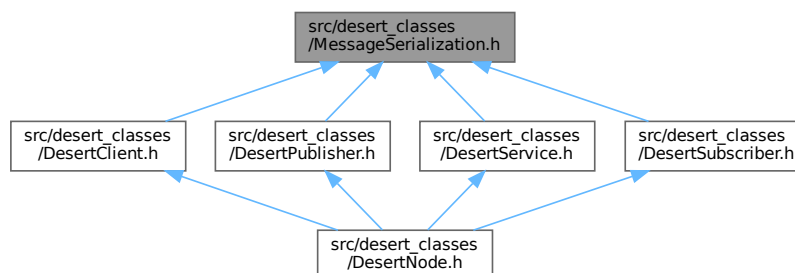
#include "CBorStream.h"
#include "CStringHelper.h"
#include "macros.h"

```

Include dependency graph for MessageSerialization.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [MessageSerialization](#)
Namespace containing serialization functions.

Macros

- `#define INTROSPECTION_C_MEMBER` `rosidl_typesupport_introspection_c__MessageMember`
- `#define INTROSPECTION_CPP_MEMBER` `rosidl_typesupport_introspection_cpp::MessageMember`
- `#define INTROSPECTION_C_MEMBERS` `rosidl_typesupport_introspection_c__MessageMembers`
- `#define INTROSPECTION_CPP_MEMBERS` `rosidl_typesupport_introspection_cpp::MessageMembers`
- `#define INTROSPECTION_C_SERVICE_MEMBERS` `rosidl_typesupport_introspection_c__ServiceMembers`
- `#define INTROSPECTION_CPP_SERVICE_MEMBERS` `rosidl_typesupport_introspection_cpp::ServiceMembers`

Functions

- `template<typename T>`
`void MessageSerialization::serialize\_field (const INTROSPECTION_CPP_MEMBER *member, void *field, cbor::TxStream &stream)`
Serialize a C++ field.
- `template<typename T>`
`void MessageSerialization::serialize\_field (const INTROSPECTION_C_MEMBER *member, void *field, cbor::TxStream &stream)`
Serialize a C field.
- `template<typename MembersType>`
`void MessageSerialization::serialize (const void *msg, const MembersType *casted_members, cbor::TxStream &stream)`
Serialize a ROS message, request or response.
- `template<typename T>`
`void MessageSerialization::deserialize\_field (const INTROSPECTION_CPP_MEMBER *member, void *field, cbor::RxStream &stream)`
Deserialize a C++ field.
- `template<typename T>`
`void MessageSerialization::deserialize\_field (const INTROSPECTION_C_MEMBER *member, void *field, cbor::RxStream &stream)`
Deserialize a C field.
- `template<typename MembersType>`
`void MessageSerialization::deserialize (void *msg, const MembersType *casted_members, cbor::RxStream &stream)`
Deserialize a ROS message, request or response.

7.25.1 Detailed Description

Namespace containing serialization functions.

The message data structure coming from upper layers is interpreted using type support informations passed by ROS2 during the creation of publishers, subscribers, clients and services. Those functions are used to compute the exact position that every data type must assume in memory and then calls `TxStream` or `RxStream` to receive or write them in the assigned location.

Author

Prof. Davide Costa

7.26 MessageSerialization.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00033
00034 #ifndef MESSAGE_SERIALIZATION_H_
00035 #define MESSAGE_SERIALIZATION_H_
00036
00037 #include "CBorStream.h"
00038 #include "CStringHelper.h"
00039 #include "macros.h"
00040
00042
00043 #include <vector>
00044 #include <string>
00045
00047
00048 #define INTROSPECTION_C_MEMBER rosidl_typesupport_introspection_c__MessageMember
00049 #define INTROSPECTION_CPP_MEMBER rosidl_typesupport_introspection_cpp::MessageMember
00050
00051 #define INTROSPECTION_C_MEMBERS rosidl_typesupport_introspection_c__MessageMembers
00052 #define INTROSPECTION_CPP_MEMBERS rosidl_typesupport_introspection_cpp::MessageMembers
00053
00054 #define INTROSPECTION_C_SERVICE_MEMBERS rosidl_typesupport_introspection_c__ServiceMembers
00055 #define INTROSPECTION_CPP_SERVICE_MEMBERS rosidl_typesupport_introspection_cpp::ServiceMembers
00056
00067 namespace MessageSerialization
00068 {
00069
00081 template<typename T>
00082 void serialize_field(const INTROSPECTION_CPP_MEMBER * member, void * field, cbor::TxStream & stream)
00083 {
00084     if (!member->is_array_)
00085     {
00086         stream << * static_cast<T *>(field);
00087     }
00088     else if (member->array_size_ && !member->is_upper_bound_)
00089     {
00090         stream.serialize_sequence(static_cast<T *>(field), member->array_size_);
00091     }
00092     else
00093     {
00094         std::vector<T> & data = *reinterpret_cast<std::vector<T> *>(field);
00095         stream << data;
00096     }
00097 }
00098
00110 template<typename T>
00111 void serialize_field(const INTROSPECTION_C_MEMBER * member, void * field, cbor::TxStream & stream)
00112 {
00113     // String specific implementation
00114     if constexpr(std::is_same_v<T, std::string>)
00115     {
00116         if (!member->is_array_)
00117         {
00118             stream << CStringHelper::convert_to_std_string(field);
00119         }
00120         else if (member->array_size_ && !member->is_upper_bound_)
00121         {
00122             stream << CStringHelper::convert_to_std_vector_string(field, member->array_size_);
00123         }
00124         else
00125         {
00126             printf("WARNING: non-fixed size sequences are currently sperimental\n");
00127             stream << CStringHelper::convert_sequence_to_std_vector_string(field);
00128         }
00129     }

```

```

00130 // U16string specific implementation
00131 else if constexpr(std::is_same_v<T, std::u16string>)
00132 {
00133     if (!member->is_array_)
00134     {
00135         stream << CStringHelper::convert_to_std_u16string(field);
00136     }
00137     else if (member->array_size_ && !member->is_upper_bound_)
00138     {
00139         stream << CStringHelper::convert_to_std_vector_u16string(field, member->array_size_);
00140     }
00141     else
00142     {
00143         printf("WARNING: non-fixed size sequences are currently sperimental\n");
00144         stream << CStringHelper::convert_sequence_to_std_vector_u16string(field);
00145     }
00146 }
00147 // Generic implementation
00148 else
00149 {
00150     if (!member->is_array_)
00151     {
00152         stream << * static_cast<T *>(field);
00153     }
00154     else if (member->array_size_ && !member->is_upper_bound_)
00155     {
00156         stream.serialize_sequence(static_cast<T *>(field), member->array_size_);
00157     }
00158     else
00159     {
00160         printf("WARNING: non-fixed size sequences are currently sperimental\n");
00161         auto & data = *reinterpret_cast<typename GenericCSequence<T>::type *>(field);
00162         // Serialize length
00163         stream << (uint32_t) data.size;
00164         stream.serialize_sequence(reinterpret_cast<T *>(data.data), data.size);
00165     }
00166 }
00167 }
00168 }
00169 }
00170
00171 template<typename MembersType>
00172 void serialize(const void * msg, const MembersType * casted_members, cbor::TxStream & stream)
00173 {
00174     for (uint32_t i = 0; i < casted_members->member_count_; ++i) {
00175         const auto member = casted_members->members_ + i;
00176         void * field = const_cast<char *>(static_cast<const char *>(msg)) + member->offset_;
00177         switch (member->type_id_) {
00178             case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_MESSAGE:
00179             {
00180                 auto sub_members = static_cast<const MembersType *>(member->members_->data);
00181                 if (!member->is_array_) {
00182                     serialize(field, sub_members, stream);
00183                 }
00184                 else if (member->array_size_ && !member->is_upper_bound_)
00185                 {
00186                     for (size_t index = 0; index < member->array_size_; ++index) {
00187                         serialize(member->get_function(field, index), sub_members, stream);
00188                     }
00189                 }
00190                 else
00191                 {
00192                     size_t array_size = member->size_function(field);
00193                     if (member->is_upper_bound_ && array_size > member->array_size_)
00194                     {
00195                         throw std::runtime_error("Sequence overcomes the maximum length");
00196                     }
00197                     // Serialize length
00198                     stream << (uint32_t) array_size;
00199                     for (size_t index = 0; index < array_size; ++index) {
00200                         serialize(member->get_function(field, index), sub_members, stream);
00201                     }
00202                 }
00203             }
00204             break;
00205             case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_BOOLEAN:
00206             {
00207                 if (!member->is_array_)
00208                 {
00209                     // Don't cast to bool here because if the bool is uninitialized the random value can't be
00210                     deserialized
00211                     stream << (*static_cast<uint8_t *>(field) ? true : false);
00212                 }
00213                 else
00214                 {
00215

```

```

00228         serialize_field<bool>(member, field, stream);
00229     }
00230     break;
00231     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_OCTET:
00232         //throw std::runtime_error("OCTET type unsupported");
00233         break;
00234     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_UINT8:
00235         serialize_field<uint8_t>(member, field, stream);
00236         break;
00237     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_CHAR:
00238         serialize_field<char>(member, field, stream);
00239         break;
00240     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_INT8:
00241         serialize_field<int8_t>(member, field, stream);
00242         break;
00243     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_FLOAT:
00244         serialize_field<float>(member, field, stream);
00245         break;
00246     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_DOUBLE:
00247         serialize_field<double>(member, field, stream);
00248         break;
00249     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_INT16:
00250         serialize_field<int16_t>(member, field, stream);
00251         break;
00252     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_UINT16:
00253         serialize_field<uint16_t>(member, field, stream);
00254         break;
00255     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_INT32:
00256         serialize_field<int32_t>(member, field, stream);
00257         break;
00258     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_UINT32:
00259         serialize_field<uint32_t>(member, field, stream);
00260         break;
00261     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_INT64:
00262         serialize_field<int64_t>(member, field, stream);
00263         break;
00264     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_UINT64:
00265         serialize_field<uint64_t>(member, field, stream);
00266         break;
00267     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_STRING:
00268         serialize_field<std::string>(member, field, stream);
00269         break;
00270     case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_WSTRING:
00271         serialize_field<std::wstring>(member, field, stream);
00272         break;
00273     default:
00274         throw std::runtime_error("unknown type");
00275     }
00276 }
00277 }
00278
00290 template<typename T>
00291 void deserialize_field(const INTROSPECTION_CPP_MEMBER * member, void * field, cbor::RxStream &
stream)
00292 {
00293     if (!member->is_array_) {
00294         stream » *static_cast<T *>(field);
00295     }
00296     else if (member->array_size_ && !member->is_upper_bound_)
00297     {
00298         stream.deserialize_sequence(static_cast<T *>(field), member->array_size_);
00299     }
00300     else
00301     {
00302         auto & vector = *reinterpret_cast<std::vector<T *>(field);
00303         new(&vector) std::vector<T>;
00304         stream » vector;
00305     }
00306 }
00307
00319 template<typename T>
00320 void deserialize_field(const INTROSPECTION_C_MEMBER * member, void * field, cbor::RxStream & stream)
00321 {
00322     // String specific implementation
00323     if constexpr (std::is_same_v<T, std::string>)
00324     {
00325         if (!member->is_array_)
00326         {
00327             std::string str;
00328             stream » str;
00329             CStringHelper::assign_string(str, field);
00330         }
00331         else if (member->array_size_ && !member->is_upper_bound_)
00332         {
00333             std::vector<std::string> cpp_string_vector;
00334             stream » cpp_string_vector;
00335         }
00336     }

```

```

00336         CStringHelper::assign_vector_string(cpp_string_vector, field, member->array_size_);
00337     }
00338     else
00339     {
00340         printf("WARNING: non-fixed size sequences are currently sperimental\n");
00341         std::vector<std::string> cpp_string_vector;
00342         stream » cpp_string_vector;
00343
00344         CStringHelper::assign_vector_string_to_sequence(cpp_string_vector, field);
00345     }
00346 }
00347 // U16string specific implementation
00348 else if constexpr(std::is_same_v<T, std::u16string>)
00349 {
00350     if (!member->is_array_)
00351     {
00352         std::u16string str;
00353         stream » str;
00354         CStringHelper::assign_u16string(str, field);
00355     }
00356     else if (member->array_size_ && !member->is_upper_bound_)
00357     {
00358         std::vector<std::u16string> cpp_string_vector;
00359         stream » cpp_string_vector;
00360
00361         CStringHelper::assign_vector_u16string(cpp_string_vector, field, member->array_size_);
00362     }
00363     else
00364     {
00365         printf("WARNING: non-fixed size sequences are currently sperimental\n");
00366         std::vector<std::u16string> cpp_string_vector;
00367         stream » cpp_string_vector;
00368
00369         CStringHelper::assign_vector_u16string_to_sequence(cpp_string_vector, field);
00370     }
00371 }
00372 // Generic implementation
00373 else
00374 {
00375     if (!member->is_array_)
00376     {
00377         stream » * static_cast<T *>(field);
00378     }
00379     else if (member->array_size_ && !member->is_upper_bound_)
00380     {
00381         stream.deserialize_sequence(static_cast<T *>(field), member->array_size_);
00382     }
00383     else
00384     {
00385         printf("WARNING: non-fixed size sequences are currently sperimental\n");
00386         auto & data = *reinterpret_cast<typename GenericCSequence<T>::type *>(field);
00387         uint32_t size = 0;
00388         stream » size;
00389         size_t dsize = static_cast<size_t>(size);
00390
00391         if (!GenericCSequence<T>::init(&data, dsize))
00392         {
00393             throw std::runtime_error("unable to initialize GenericCSequence");
00394         }
00395
00396         stream.deserialize_sequence(reinterpret_cast<T *>(data.data), dsize);
00397     }
00398 }
00399 }
00400
00401 template<typename MembersType>
00402 void deserialize(void * msg, const MembersType * casted_members, cbor::RxStream & stream)
00403 {
00404     for (uint32_t i = 0; i < casted_members->member_count_; ++i) {
00405         const auto member = casted_members->members_ + i;
00406         void * field = static_cast<char *>(msg) + member->offset_;
00407         switch (member->type_id_) {
00408             case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_MESSAGE:
00409             {
00410                 auto sub_members = static_cast<const MembersType *>(member->members_->data);
00411                 if (!member->is_array_) {
00412                     deserialize(field, sub_members, stream);
00413                 }
00414                 else if (member->array_size_ && !member->is_upper_bound_)
00415                 {
00416                     for (size_t index = 0; index < member->array_size_; ++index) {
00417                         deserialize(member->get_function(field, index), sub_members, stream);
00418                     }
00419                 }
00420                 else
00421                 {
00422                     // Deserialize length

```

```

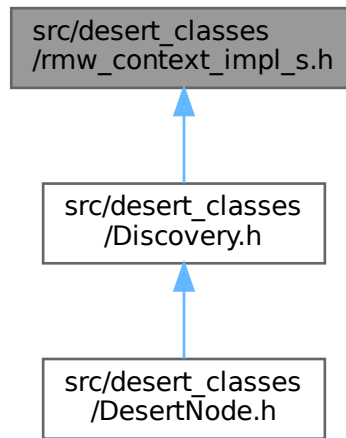
00436         uint32_t array_size = 0;
00437         stream » array_size;
00438
00439         auto vector = reinterpret_cast<std::vector<unsigned char>*>(field);
00440         new(vector) std::vector<unsigned char>;
00441         member->resize_function(field, array_size);
00442
00443         for (size_t index = 0; index < array_size; ++index) {
00444             deserialize(member->get_function(field, index), sub_members, stream);
00445         }
00446     }
00447 }
00448 break;
00449 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_BOOLEAN:
00450     deserialize_field<bool>(member, field, stream);
00451     break;
00452 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_OCTET:
00453     //throw std::runtime_error("OCTET type unsupported");
00454     break;
00455 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_UINT8:
00456     deserialize_field<uint8_t>(member, field, stream);
00457     break;
00458 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_CHAR:
00459     deserialize_field<char>(member, field, stream);
00460     break;
00461 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_INT8:
00462     deserialize_field<int8_t>(member, field, stream);
00463     break;
00464 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_FLOAT:
00465     deserialize_field<float>(member, field, stream);
00466     break;
00467 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_DOUBLE:
00468     deserialize_field<double>(member, field, stream);
00469     break;
00470 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_INT16:
00471     deserialize_field<int16_t>(member, field, stream);
00472     break;
00473 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_UINT16:
00474     deserialize_field<uint16_t>(member, field, stream);
00475     break;
00476 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_INT32:
00477     deserialize_field<int32_t>(member, field, stream);
00478     break;
00479 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_UINT32:
00480     deserialize_field<uint32_t>(member, field, stream);
00481     break;
00482 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_INT64:
00483     deserialize_field<int64_t>(member, field, stream);
00484     break;
00485 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_UINT64:
00486     deserialize_field<uint64_t>(member, field, stream);
00487     break;
00488 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_STRING:
00489     deserialize_field<std::string>(member, field, stream);
00490     break;
00491 case ::rosidl_typesupport_introspection_cpp::ROS_TYPE_WSTRING:
00492     deserialize_field<std::wstring>(member, field, stream);
00493     break;
00494 default:
00495     throw std::runtime_error("unknown type");
00496 }
00497 }
00498 }
00499
00500 }
00501
00502
00503 #endif

```

7.27 src/desert_classes/rmw_context_impl_s.h File Reference

Implementation for the context variable.

This graph shows which files directly or indirectly include this file:



Classes

- struct [rmw_context_impl_s](#)

7.27.1 Detailed Description

Implementation for the context variable.

Context is used to store informations about the current network structure using variables included from `rmw_dds_common`. This struct provides an implementation for `rmw_context_impl_t` and it must be present to avoid compile errors.

Author

Prof. Davide Costa

7.28 rmw_context_impl_s.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
  
```

```

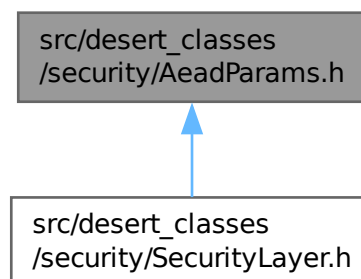
00015  *
00016  *   You should have received a copy of the GNU General Public License
00017  *   along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *
00019  *****/
00031
00033
00034 #include "rcpputils/scope_exit.hpp"
00035 #include "rmw_dds_common/context.hpp"
00036 #include "rmw_dds_common/graph_cache.hpp"
00037 #include "rmw_dds_common/msg/participant_entities_info.hpp"
00038 #include "rmw_dds_common/qos.hpp"
00039
00041
00042 #ifndef RMW_CONTEXT_IMPL_H_
00043 #define RMW_CONTEXT_IMPL_H_
00044
00045 struct rmw_context_impl_s
00046 {
00047     rmw_dds_common::Context common;
00048     bool is_shutdown{false};
00049
00050     rmw_context_impl_s()
00051     : common()
00052     {
00053         /* destructor relies on these being initialized properly */
00054         common.thread_is_running.store(false);
00055         common.graph_guard_condition = nullptr;
00056         common.pub = nullptr;
00057         common.sub = nullptr;
00058     }
00059
00060     ~rmw_context_impl_s()
00061     {
00062     }
00063 };
00064
00065 #endif

```

7.29 src/desert_classes/security/AeadParams.h File Reference

Class containing the parameters of the selected cryptography.

This graph shows which files directly or indirectly include this file:



Classes

- class [security::AeadParams](#)

Macros

- `#define AEAD_ALGO_ASCON_128 "Ascon-AEAD128"`
- `#define AEAD_ALGO_ASCON_128_64 "Ascon-AEAD128-64"`
- `#define AEAD_ALGO_ASCON_128_32 "Ascon-AEAD128-32"`
- `#define KDF_ALGO_ASCON_256 "HKDF-Ascon256"`

Enumerations

- `enum AeadAlgorithms {`
`ASCON_AEAD128 , ASCON_AEAD128_64 , ASCON_AEAD128_32 , A128GCM ,`
`AEAD_UNKNOWN }`
- `enum KdfAlgorithms { HKDF_ASCON , HKDF_HMAC256 , KDF_UNKNOWN }`

7.29.1 Detailed Description

Class containing the parameters of the selected cryptography.

This header defines the algorithms and parameter sets used by the security layer. It provides AEAD and KDF identifiers together with the key, nonce, and authentication-tag sizes required by each supported AEAD algorithm. These parameters are used to configure cryptographic context derivation.

Author

Dmytro Ochkas

7.30 AeadParams.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2026 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00020
00021
00022
00023 #ifndef SECURITY_CONFIG_H
00024 #define SECURITY_CONFIG_H
00025
00026
00027
00028 #include <stdint>
00029
00030
00031
00032
00033 #define AEAD_ALGO_ASCON_128 "Ascon-AEAD128"
00034 #define AEAD_ALGO_ASCON_128_64 "Ascon-AEAD128-64"
00035 #define AEAD_ALGO_ASCON_128_32 "Ascon-AEAD128-32"
00036
00037 #define KDF_ALGO_ASCON_256 "HKDF-Ascon256"
00038
00039 namespace security
00040 {

```

```

00050
00051 enum AeadAlgorithms
00052 {
00053     ASCON_AEAD128,
00054     ASCON_AEAD128_64,
00055     ASCON_AEAD128_32,
00056     A128GCM,
00057     AEAD_UNKNOWN
00058 };
00059
00060 enum KdfAlgorithms
00061 {
00062     HKDF_ASCON,
00063     HKDF_HMAC256,
00064     KDF_UNKNOWN
00065 };
00066
00067 class AeadParams
00068 {
00069     public:
00075     AeadParams(AeadAlgorithms alg);
00076
00082     AeadAlgorithms get_alg() const { return alg_; }
00083
00089     uint16_t get_key_size() const { return key_size_; }
00090
00096     uint16_t get_nonce_size() const { return nonce_size_; }
00097
00103     uint16_t get_tag_size() const { return tag_size_; }
00104
00105     private:
00106     AeadAlgorithms alg_;
00107     uint16_t key_size_;
00108     uint16_t nonce_size_;
00109     uint16_t tag_size_;
00110 };
00111
00112 } // namespace security
00113
00114 #endif

```

7.31 src/desert_classes/security/sec_utils.h File Reference

Utilities for the security layer.

Macros

- #define [MIN](#)(a, b)

Functions

- bool [decode_hex](#) (const std::string &hex, std::vector< uint8_t > &out)
Decodes a hexadecimal string into a byte vector.

7.31.1 Detailed Description

Utilities for the security layer.

This header declares utility functions shared by the security layer. It includes helpers for converting hexadecimal strings into byte vectors and small generic macros used while processing cryptographic input data. These utilities support configuration parsing and internal data handling.

Author

Dmytro Ochkas

7.31.2 Macro Definition Documentation

7.31.2.1 MIN

```
#define MIN(
    a,
    b)
```

Value:

```
((a) > (b) ? (b) : (a))
```

7.31.3 Function Documentation

7.31.3.1 decode_hex()

```
bool decode_hex (
    const std::string & hex,
    std::vector< uint8_t > & out)
```

Decodes a hexadecimal string into a byte vector.

The input can optionally begin with the prefixes `0x` or `0X`. After an optional prefix, the string must contain an even number of valid hexadecimal characters.

Parameters

<i>hex</i>	String containing hexadecimal byte values.
<i>out</i>	Output vector receiving the decoded bytes.

Returns

true if the complete input is valid and has been decoded; false if the input has an odd length or contains non-hexadecimal characters.

Note

On failure, `out` is cleared.

7.32 sec_utils.h

[Go to the documentation of this file.](#)

```
00001 /*****
00002  * Copyright (C) 2026 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
```

```

00013 *   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the      *
00014 *   GNU General Public License for more details.                        *
00015 *                                                                       *
00016 *   You should have received a copy of the GNU General Public License   *
00017 *   along with RMW desert.  If not, see <http://www.gnu.org/licenses/>.  *
00018 *   *****/
00019
00032
00033 #ifndef SEC_UTILS_H
00034 #define SEC_UTILS_H
00035
00037
00038 #include <string>
00039 #include <vector>
00040 #include <stdint>
00041
00043
00044 #define MIN(a, b) ((a) > (b) ? (b) : (a))
00045
00061 bool decode_hex(const std::string& hex, std::vector<uint8_t>& out);
00062
00063 #endif

```

7.33 src/desert_classes/security/SecurityLayer.h File Reference

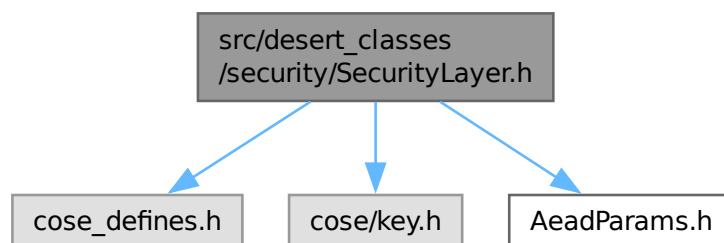
Class used to handle the encryption of the stream.

```

#include <cose_defines.h>
#include <cose/key.h>
#include "AeadParams.h"

```

Include dependency graph for SecurityLayer.h:



Classes

- class [security::SecurityLayer](#)

Macros

- #define `DEFAULT_MASTER_SALT_LEN` 16
- #define `COSE_SERIALIZATION_MAX_OVERHEAD` 12
- #define `COSE_INTERNAL_BUF_SIZE` 512

Enumerations

- enum **SecurityResult** {
OK = 0 , **PARAM_ERROR** = -1 , **KEY_SIZE_ERROR** = -2 , **KEY_ENV_ERROR** = -3 ,
NONCE_ENV_ERROR = -4 , **NONCE_SIZE_ERROR** = -5 , **BUFFER_ERROR** = -6 , **CONTEXT_DERIVE**
_ERROR = -7 ,
COMP_ERROR = -8 , **WRAP_ERROR** = -100 , **UNWRAP_ERROR** = -200 , **INTERNAL_ERROR** = -999 }

7.33.1 Detailed Description

Class used to handle the encryption of the stream.

This header declares the security layer used to protect application data. It derives sender and receiver contexts, builds nonces, and performs COSE wrapping and unwrapping of payloads using the selected AEAD and KDF schemes. It also manages key material, identifiers, and serialization buffers.

Author

Dmytro Ochkas

7.34 SecurityLayer.h

[Go to the documentation of this file.](#)

```
00001 /*****
00002  * Copyright (C) 2026 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00020
00021
00022
00023 #ifndef SECURITY_LAYER_H
00024 #define SECURITY_LAYER_H
00025
00026
00027
00028 #include <memory>
00029 #include <string>
00030 #include <vector>
00031
00032
00033
00034 #include <cose_defines.h>
00035 #include <cose/key.h>
00036
00037 #include "AeadParams.h"
00038
00039 #define DEFAULT_MASTER_SALT_LEN 16
00040 #define COSE_SERIALIZATION_MAX_OVERHEAD 12
00041 #define COSE_INTERNAL_BUF_SIZE 512
00042
00043 namespace security
00044 {
00045
00046 enum SecurityResult
00047 {
00048     OK = 0,
00049     PARAM_ERROR = -1,
00050     KEY_SIZE_ERROR = -2,
```

```

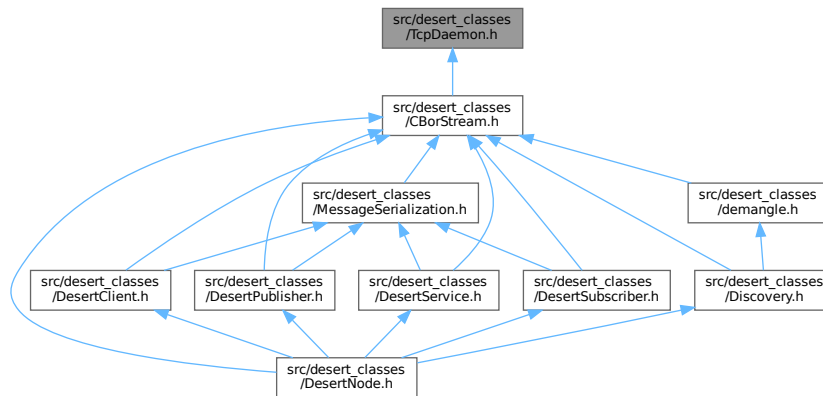
00061 KEY_ENV_ERROR = -3,
00062 NONCE_ENV_ERROR = -4,
00063 NONCE_SIZE_ERROR = -5,
00064 BUFFER_ERROR = -6,
00065 CONTEXT_DERIVE_ERROR = -7,
00066 COMP_ERROR = -8,
00067 WRAP_ERROR = -100,
00068 UNWRAP_ERROR = -200,
00069 INTERNAL_ERROR = -999
00070 };
00071
00072 class SecurityLayer
00073 {
00074 public:
00081     SecurityLayer();
00082
00096     SecurityLayer(const AeadParams &aead_params, KdfAlgorithms kdf, const std::vector<uint8_t>
&master_key,
00097         const std::vector<uint8_t> &master_salt, size_t piv_size, size_t sender_seq_number,
00098         const std::vector<uint8_t> &sender_id, const std::vector<uint8_t> &receiver_id);
00099
00119     SecurityResult wrap(uint8_t* data, size_t data_len, size_t data_max_len, uint8_t** cose_ptr,
size_t* cose_len);
00120
00136     SecurityResult unwrap(uint8_t* data, size_t data_len, size_t data_max_len, size_t* new_data_len);
00137
00138 private:
00139     SecurityResult get_master_key_env(std::vector<uint8_t>& master_key) const;
00140     SecurityResult generate_nonce();
00141     SecurityResult build_kdf_info(const std::vector<uint8_t>& id, cose_algo_t alg, const std::string&
type, size_t len, uint8_t** out, size_t* out_len);
00142     SecurityResult derive_key(const std::vector<uint8_t>& ikm, const std::vector<uint8_t>& salt, const
std::vector<uint8_t>& id, cose_algo_t algo, std::vector<uint8_t>& key_out, cose_key_t* cose_k_out);
00143     SecurityResult derive_iv(const std::vector<uint8_t>& ikm, const std::vector<uint8_t>& salt,
cose_algo_t alg, std::vector<uint8_t>& iv_out);
00144     SecurityResult derive_context(const std::vector<uint8_t>& master_key, const std::vector<uint8_t>&
master_salt);
00145     SecurityResult cose_stateless_compress(uint8_t* cose, size_t cose_len, uint8_t** out, size_t*
out_len) const;
00146     SecurityResult cose_stateless_decompress(uint8_t* data, size_t data_len, size_t data_max_len,
size_t* out_len) const;
00147
00148     /* Algorithms to use */
00149     AeadParams aead_params_;
00150     KdfAlgorithms kdf_;
00151     /* ***** */
00152
00153     /* Nonce-related fields */
00154     std::vector<uint8_t> context_iv_{};
00155     size_t piv_size_;
00156     std::vector<uint8_t> piv_bytes_{};
00157     size_t sender_seq_number_;
00158     /* ***** */
00159
00160     /* Key-related fields */
00161     std::vector<uint8_t> sender_id_{};
00162     std::vector<uint8_t> sender_key_bytes_{};
00163     cose_key_t sender_cose_key_{};
00164     std::vector<uint8_t> receiver_id_{};
00165     std::vector<uint8_t> receiver_key_bytes_{};
00166     cose_key_t receiver_cose_key_{};
00167     /* ***** */
00168
00169     uint8_t internal_buf_[COSE_INTERNAL_BUF_SIZE]{};
00170 };
00171
00172 } // namespace security
00173
00174 #endif

```

7.35 src/desert_classes/TcpDaemon.h File Reference

Class used to send and receive data from the DESERT socket.

This graph shows which files directly or indirectly include this file:



Classes

- class [TcpDaemon](#)

Macros

- #define **MAX_PACKET_LENGTH** 512
- #define **ADDRESS** "127.0.0.1"
- #define **START_MARKER** 0b10011001
- #define **END_MARKER** 0b01010101
- #define **BYTE_MASK** 0b11111111

7.35.1 Detailed Description

Class used to send and receive data from the DESERT socket.

The DESERT protocol stack interacts with the application level through a socket, used to send and receive a binary stream containing packets. This class connects to the socket and creates two threads, that run continuously to store and send packets in the static members rx_packets and tx_packets

Author

Prof. Davide Costa

7.36 TcpDaemon.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00032
00033 #ifndef TCP_DAEMON_H_
00034 #define TCP_DAEMON_H_
00035
00037
00038 #include <queue>
00039 #include <vector>
00040 #include <cstdint>
00041 #include <cstdio>
00042 #include <cstring>
00043 #include <thread>
00044 #include <chrono>
00045
00046 #include <arpa/inet.h>
00047 #include <sys/socket.h>
00048 #include <sys/poll.h>
00049 #include <unistd.h>
00050
00051 #include "rmw/error_handling.h"
00052
00054
00055 #define MAX_PACKET_LENGTH 512
00056
00057 #define ADDRESS "127.0.0.1"
00058
00059 #define START_MARKER 0b10011001
00060 #define END_MARKER 0b01010101
00061 #define BYTE_MASK 0b11111111
00062
00063 class TcpDaemon
00064 {
00065 public:
00066     TcpDaemon();
00067
00076     bool init(int port);
00085     static std::vector<uint8_t> read_packet();
00094     static void enqueue_packet(std::vector<uint8_t> packet);
00095
00096 private:
00097     static int _client_fd;
00098     static std::queue<std::vector<uint8_t> > _rx_packets;
00099     static std::queue<std::vector<uint8_t> > _tx_packets;
00100
00102     void socket_rx_communication();
00103     void socket_tx_communication();
00104
00105 };
00106
00107 #endif

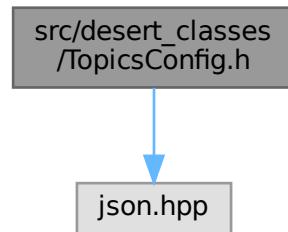
```

7.37 src/desert_classes/TopicsConfig.h File Reference

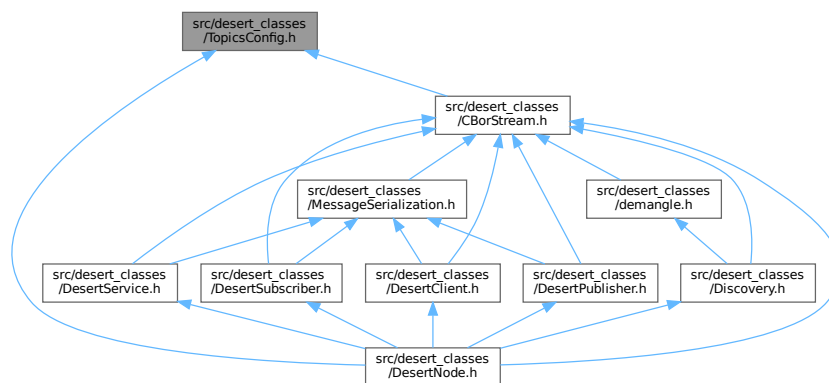
Class used to store configurations.

```
#include "json.hpp"
```

Include dependency graph for TopicsConfig.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [TopicsConfig](#)

7.37.1 Detailed Description

Class used to store configurations.

In order to prevent sending a string containing the topic name for each packet, a configuration file is used to associate each topic string to an integer that uses much less bandwidth and blocks topics that are not in this list.

Author

Prof. Davide Costa

7.38 TopicsConfig.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * Copyright (C) 2024 Davide Costa
00003  *
00004  * This file is part of RMW desert.
00005  *
00006  * RMW desert is free software: you can redistribute it and/or modify it
00007  * under the terms of the GNU General Public License as published by the
00008  * Free Software Foundation, either version 3 of the License, or any
00009  * later version.
00010  *
00011  * RMW desert is distributed in the hope that it will be useful,
00012  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00013  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014  * GNU General Public License for more details.
00015  *
00016  * You should have received a copy of the GNU General Public License
00017  * along with RMW desert. If not, see <http://www.gnu.org/licenses/>.
00018  *****/
00019
00031
00032 #ifndef TOPICS_CONFIG_H_
00033 #define TOPICS_CONFIG_H_
00034
00036
00037 #include <map>
00038 #include <string>
00039 #include <cstdint>
00040 #include <fstream>
00041 #include <cstdlib>
00042
00044
00045 #include "json.hpp"
00046
00047 using namespace nllohmann::json_abi_v3_l1_3;
00048
00049 class TopicsConfig
00050 {
00051 public:
00052
00061     static void load_configuration();
00070     static uint8_t get_topic_identifier(std::string name);
00079     static std::string get_identifier_topic(uint8_t identifier);
00080
00081 private:
00082     static std::map<std::string, uint8_t> _topics_list;
00083     static std::map<uint8_t, std::string> _identifiers_list;
00084 };
00085
00086 #endif // MACROS_HPP_

```

Index

- add_client
 - DesertNode, [33](#)
- add_publisher
 - DesertNode, [33](#)
- add_service
 - DesertNode, [33](#)
- add_subscriber
 - DesertNode, [33](#)
- AeadParams
 - security::AeadParams, [25](#)
- assign_string
 - CStringHelper, [10](#)
- assign_u16string
 - CStringHelper, [10](#)
- assign_vector_string
 - CStringHelper, [10](#)
- assign_vector_string_to_sequence
 - CStringHelper, [11](#)
- assign_vector_u16string
 - CStringHelper, [11](#)
- assign_vector_u16string_to_sequence
 - CStringHelper, [12](#)
- cbor::RxStream, [44](#)
 - clear_buffer, [46](#)
 - data_available, [46](#)
 - deserialize_sequence, [46](#)
 - get_identifier, [46](#)
 - get_name, [46](#)
 - get_type, [47](#)
 - interpret_packets, [47](#)
 - operator>>, [47–51](#)
 - push_packet, [51](#)
 - RxStream, [45](#)
- cbor::RxStream::ITEM, [43](#)
- cbor::TxStream, [57](#)
 - end_transmission, [58](#)
 - operator<<, [58–62](#)
 - serialize_sequence, [62](#)
 - start_transmission, [62, 63](#)
 - TxStream, [58](#)
- cbor::TxStream::ERROR, [43](#)
- cbor::TxStream::WRITER, [63](#)
- CircularQueue< T, MaxLen, Container >, [27](#)
- clear_buffer
 - cbor::RxStream, [46](#)
- convert_sequence_to_std_vector_string
 - CStringHelper, [12](#)
- convert_sequence_to_std_vector_u16string
 - CStringHelper, [12](#)
- convert_to_std_string
 - CStringHelper, [13](#)
- convert_to_std_u16string
 - CStringHelper, [13](#)
- convert_to_std_vector_string
 - CStringHelper, [13](#)
- convert_to_std_vector_u16string
 - CStringHelper, [14](#)
- CStringHelper, [9](#)
 - assign_string, [10](#)
 - assign_u16string, [10](#)
 - assign_vector_string, [10](#)
 - assign_vector_string_to_sequence, [11](#)
 - assign_vector_u16string, [11](#)
 - assign_vector_u16string_to_sequence, [12](#)
 - convert_sequence_to_std_vector_string, [12](#)
 - convert_sequence_to_std_vector_u16string, [12](#)
 - convert_to_std_string, [13](#)
 - convert_to_std_u16string, [13](#)
 - convert_to_std_vector_string, [13](#)
 - convert_to_std_vector_u16string, [14](#)
- data_available
 - cbor::RxStream, [46](#)
- decode_hex
 - sec_utils.h, [103](#)
- demangle_publisher_from_topic
 - Discovery, [16](#)
- demangle_service_from_topic
 - Discovery, [16](#)
- demangle_service_reply_from_topic
 - Discovery, [16](#)
- demangle_service_request_from_topic
 - Discovery, [17](#)
- demangle_subscriber_from_topic
 - Discovery, [17](#)
- demangle_topic
 - Discovery, [17](#)
- deserialize
 - MessageSerialization, [21](#)
- deserialize_field
 - MessageSerialization, [22](#)
- deserialize_sequence
 - cbor::RxStream, [46](#)
- DesertClient, [28](#)
 - DesertClient, [28](#)
 - get_gid, [28](#)
 - get_request_type_name, [28](#)
 - get_response_type_name, [29](#)
 - get_service_name, [29](#)

- has_data, [29](#)
- read_response, [29](#)
- send_request, [30](#)
- DesertGuardCondition, [30](#)
 - get_has_triggered, [31](#)
 - has_triggered, [31](#)
 - trigger, [31](#)
- DesertNode, [32](#)
 - add_client, [33](#)
 - add_publisher, [33](#)
 - add_service, [33](#)
 - add_subscriber, [33](#)
 - DesertNode, [32](#)
 - get_gid, [34](#)
 - remove_client, [34](#)
 - remove_publisher, [34](#)
 - remove_service, [35](#)
 - remove_subscriber, [35](#)
- DesertPublisher, [36](#)
 - DesertPublisher, [36](#)
 - get_gid, [36](#)
 - get_topic_name, [36](#)
 - get_type_name, [37](#)
 - push, [37](#)
- DesertService, [38](#)
 - DesertService, [38](#)
 - get_gid, [38](#)
 - get_request_type_name, [38](#)
 - get_response_type_name, [39](#)
 - get_service_name, [39](#)
 - has_data, [39](#)
 - read_request, [39](#)
 - send_response, [40](#)
- DesertSubscriber, [40](#)
 - DesertSubscriber, [41](#)
 - get_gid, [41](#)
 - get_topic_name, [41](#)
 - get_type_name, [41](#)
 - has_data, [42](#)
 - read_data, [42](#)
- DesertWaitset, [43](#)
- Discovery, [14](#)
 - demangle_publisher_from_topic, [16](#)
 - demangle_service_from_topic, [16](#)
 - demangle_service_reply_from_topic, [16](#)
 - demangle_service_request_from_topic, [17](#)
 - demangle_subscriber_from_topic, [17](#)
 - demangle_topic, [17](#)
 - discovery_thread, [18](#)
 - discovery_thread_start, [18](#)
 - discovery_thread_stop, [18](#)
 - identity_demangle, [19](#)
 - resolve_prefix, [19](#)
 - send_discovery_beacon, [20](#)
 - send_discovery_request, [20](#)
- discovery_thread
 - Discovery, [18](#)
- discovery_thread_start
 - Discovery, [18](#)
- Discovery, [18](#)
- discovery_thread_stop
 - Discovery, [18](#)
- end_transmission
 - cbor::TxStream, [58](#)
- enqueue_packet
 - TcpDaemon, [55](#)
- GenericCSequence< T >, [43](#)
- get_alg
 - security::AeadParams, [26](#)
- get_gid
 - DesertClient, [28](#)
 - DesertNode, [34](#)
 - DesertPublisher, [36](#)
 - DesertService, [38](#)
 - DesertSubscriber, [41](#)
- get_has_triggered
 - DesertGuardCondition, [31](#)
- get_identifier
 - cbor::RxStream, [46](#)
- get_identifier_topic
 - TopicsConfig, [56](#)
- get_key_size
 - security::AeadParams, [26](#)
- get_name
 - cbor::RxStream, [46](#)
- get_nonce_size
 - security::AeadParams, [26](#)
- get_request_type_name
 - DesertClient, [28](#)
 - DesertService, [38](#)
- get_response_type_name
 - DesertClient, [29](#)
 - DesertService, [39](#)
- get_service_name
 - DesertClient, [29](#)
 - DesertService, [39](#)
- get_tag_size
 - security::AeadParams, [26](#)
- get_topic_identifier
 - TopicsConfig, [56](#)
- get_topic_name
 - DesertPublisher, [36](#)
 - DesertSubscriber, [41](#)
- get_type
 - cbor::RxStream, [47](#)
- get_type_name
 - DesertPublisher, [37](#)
 - DesertSubscriber, [41](#)
- has_data
 - DesertClient, [29](#)
 - DesertService, [39](#)
 - DesertSubscriber, [42](#)
- has_triggered
 - DesertGuardCondition, [31](#)

- identity_demangle
 - Discovery, [19](#)
- init
 - TcpDaemon, [55](#)
- interpret_packets
 - cbor::RxStream, [47](#)
- load_configuration
 - TopicsConfig, [56](#)
- macros.h
 - SPECIALIZE_GENERIC_C_SEQUENCE, [91](#)
- MessageSerialization, [21](#)
 - deserialize, [21](#)
 - deserialize_field, [22](#)
 - serialize, [23](#)
 - serialize_field, [23](#)
- MIN
 - sec_utils.h, [103](#)
- operator<<
 - cbor::TxStream, [58–62](#)
- operator>>
 - cbor::RxStream, [47–51](#)
- push
 - DesertPublisher, [37](#)
- push_packet
 - cbor::RxStream, [51](#)
- read_data
 - DesertSubscriber, [42](#)
- read_packet
 - TcpDaemon, [55](#)
- read_request
 - DesertService, [39](#)
- read_response
 - DesertClient, [29](#)
- remove_client
 - DesertNode, [34](#)
- remove_publisher
 - DesertNode, [34](#)
- remove_service
 - DesertNode, [35](#)
- remove_subscriber
 - DesertNode, [35](#)
- resolve_prefix
 - Discovery, [19](#)
- rmw_context_impl_s, [44](#)
- RxStream
 - cbor::RxStream, [45](#)
- sec_utils.h
 - decode_hex, [103](#)
 - MIN, [103](#)
- security::AeadParams, [25](#)
 - AeadParams, [25](#)
 - get_alg, [26](#)
 - get_key_size, [26](#)
 - get_nonce_size, [26](#)
 - get_tag_size, [26](#)
- security::SecurityLayer, [52](#)
 - SecurityLayer, [52](#)
 - unwrap, [53](#)
 - wrap, [53](#)
- SecurityLayer
 - security::SecurityLayer, [52](#)
- send_discovery_beacon
 - Discovery, [20](#)
- send_discovery_request
 - Discovery, [20](#)
- send_request
 - DesertClient, [30](#)
- send_response
 - DesertService, [40](#)
- serialize
 - MessageSerialization, [23](#)
- serialize_field
 - MessageSerialization, [23](#)
- serialize_sequence
 - cbor::TxStream, [62](#)
- SPECIALIZE_GENERIC_C_SEQUENCE
 - macros.h, [91](#)
- src/desert_classes/CBorStream.h, [65, 67](#)
- src/desert_classes/CStringHelper.h, [69, 71](#)
- src/desert_classes/demangle.h, [72, 74](#)
- src/desert_classes/DesertClient.h, [74, 76](#)
- src/desert_classes/DesertGuardCondition.h, [77](#)
- src/desert_classes/DesertNode.h, [78, 79](#)
- src/desert_classes/DesertPublisher.h, [80, 81](#)
- src/desert_classes/DesertService.h, [82, 83](#)
- src/desert_classes/DesertSubscriber.h, [84, 86](#)
- src/desert_classes/DesertWaitSet.h, [87](#)
- src/desert_classes/Discovery.h, [88, 89](#)
- src/desert_classes/macros.h, [90, 91](#)
- src/desert_classes/MessageSerialization.h, [92, 94](#)
- src/desert_classes/rmw_context_impl_s.h, [98, 99](#)
- src/desert_classes/security/AeadParams.h, [100, 101](#)
- src/desert_classes/security/sec_utils.h, [102, 103](#)
- src/desert_classes/security/SecurityLayer.h, [104, 105](#)
- src/desert_classes/TcpDaemon.h, [106, 108](#)
- src/desert_classes/TopicsConfig.h, [108, 110](#)
- start_transmission
 - cbor::TxStream, [62, 63](#)
- TcpDaemon, [54](#)
 - enqueue_packet, [55](#)
 - init, [55](#)
 - read_packet, [55](#)
- TopicsConfig, [56](#)
 - get_identifier_topic, [56](#)
 - get_topic_identifier, [56](#)
 - load_configuration, [56](#)
- trigger
 - DesertGuardCondition, [31](#)
- TxStream
 - cbor::TxStream, [58](#)
- unwrap

security::SecurityLayer, [53](#)

wrap

security::SecurityLayer, [53](#)